Luiss
Libera Università Internazionale
degli Studi Sociali Guido Carli

Lab on Asymmetric Cryptography & Digital Signatures

Data Privacy and Security

Data Science and Management (DASMA)

LUISS

From Symmetric to Asymmetric Cryptography

Symmetric Cryptography suffers from **two main limitations**:

1. **Key Distribution**: Alice and Bob must securely exchange a shared secret key in advance.

2. **Key Management**: when we have n users, we need to manage $n(n-1)/2$ keys:
   - There are billions of users accessing the internet, it cannot scale!
   - E.g., Whatsapp has over 2B (billion) users, and would need to manage $2B \times 2B/2 \approx 2$ quintillion keys!

From Symmetric to Asymmetric Cryptography

Symmetric Cryptography suffers from **two main limitations**:

1. **Key Distribution**: Alice and Bob must securely exchange a shared secret key in advance.

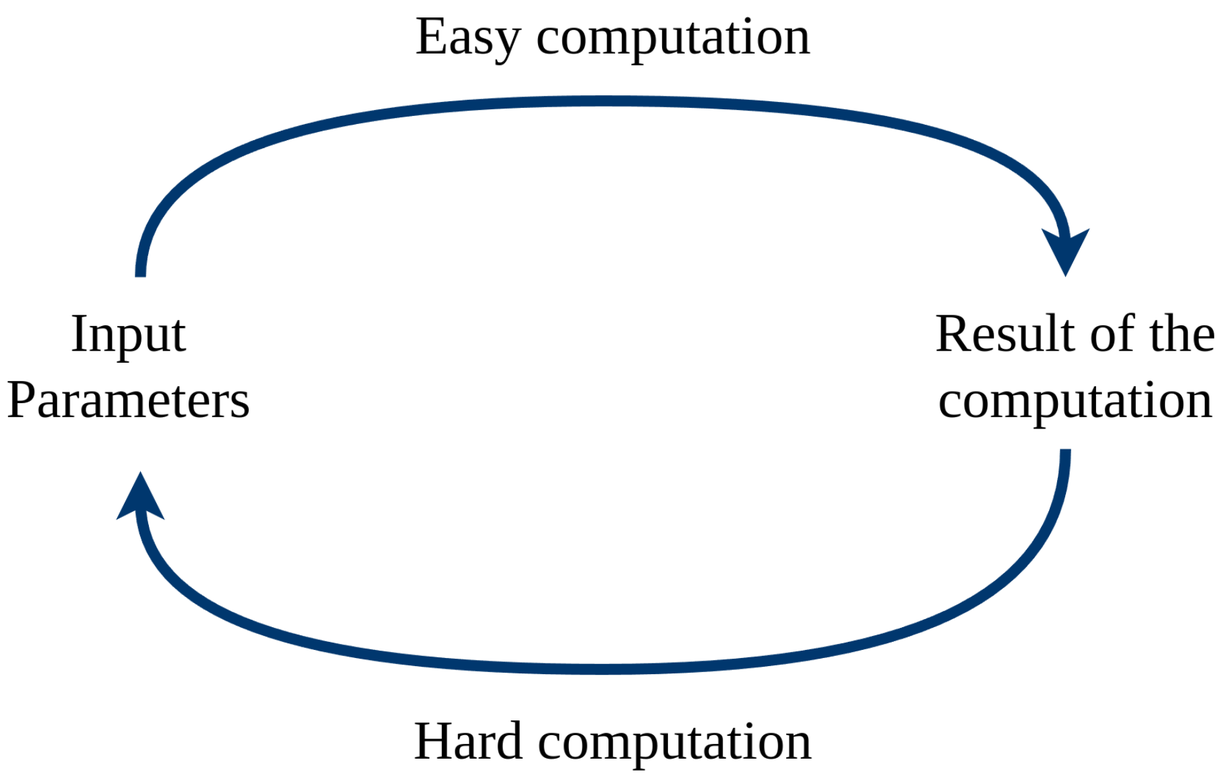
2. **Key Management**: when we have n users, we need to manage $n(n-1)/2$ keys:
   - There are billions of users accessing the internet, it cannot scale!
   - E.g., Whatsapp has over 2B (billion) users, and would need to manage $2B \times 2B/2 \approx 2$ quintillion keys!

Asymmetric cryptography addresses both issues by leveraging **mathematical asymmetry**.

Examples of asymmetric cryptographic algorithms include RSA, Diffie-Hellman, ...

How to achieve this *asymmetry*?

Most asymmetric cryptographic algorithms exploit a mathematical problem showing this behavior:



Easy to compute the result starting from the input parameters
but hard to compute the input parameters starting from the result

Diffie-Hellman Algorithm

Shared Key Exchange

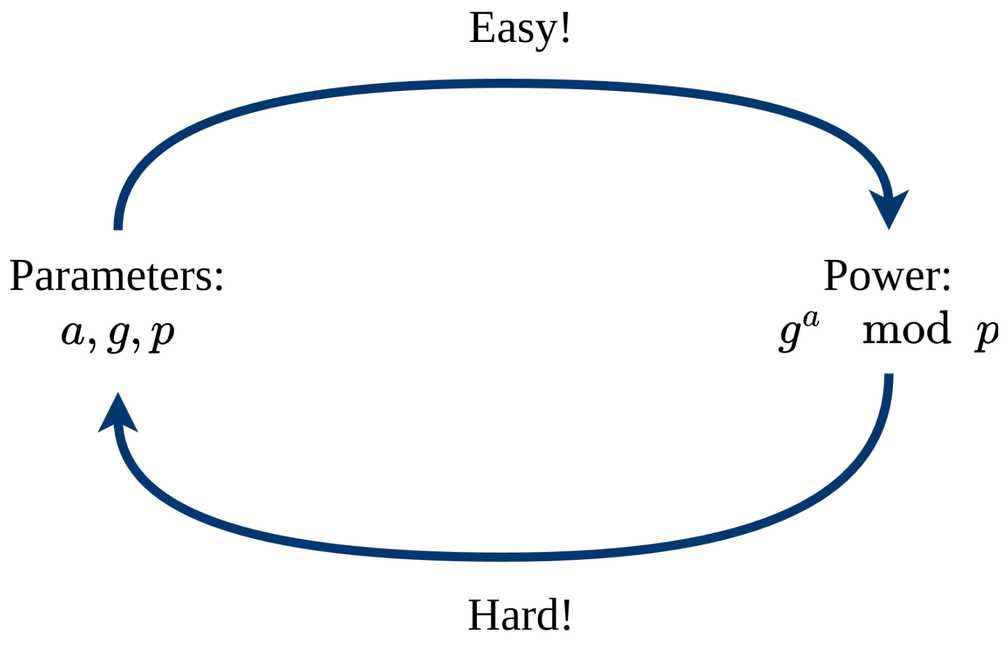
What if Alice and Bob wanted to use a **single shared secret key** for communication, without revealing it to an eavesdropper?

Shared Key Exchange

What if Alice and Bob wanted to use a **single shared secret key** for communication, without revealing it to an eavesdropper?

DH allows them to agree on a shared secret by exploiting the **Discrete Logarithm Problem** (**DLP**):

- **Easy to compute** the exponentiation of an integer modulo a prime.
- **Hard to reverse** it (through logarithms) to deduce the original exponent, which is needed to compute the shared secret.



Diffie-Hellman Key Exchange Algorithm (1976)

Public parameters:

- Prime number p
- Generator g

Diffie-Hellman Key Exchange Algorithm (1976)

Public parameters:

- Prime number p
- Generator g

Alice:

$$a \in [1, p-1]$$
$$g^a \mod p$$

Bob:

$$b \in [1, p-1]$$
$$g^b \mod p$$

Diffie-Hellman Key Exchange Algorithm (1976)

Public parameters:

- Prime number p
- Generator g

Alice:	Exchanged messages:	Bob:
$a \in [1, p-1]$		$b \in [1, p-1]$
$g^a \mod p$	$\xrightarrow{g^a \mod p}$	$g^b \mod p$
	$\xleftarrow{g^b \mod p}$	

Diffie-Hellman Key Exchange Algorithm (1976)

Public parameters:

- Prime number p
- Generator g

Alice:	Exchanged messages:	Bob:
$a \in [1, p-1]$		$b \in [1, p-1]$
$g^a \mod p$		$g^b \mod p$
	$\xrightarrow{g^a \mod p}$	
	$\xleftarrow{g^b \mod p}$	
$(g^b)^a \mod p = g^{ab} \mod p$		$(g^a)^b \mod p = g^{ab} \mod p$

NOTICE: In the end, both parties have the same shared secret key $k = g^{ab} \bmod p$.

DH in Python

```python
# Public parameters
p = 23        # prime number
g = 5         # generator

# Alice's private key
a = 6
# Alice's public key
A = pow(g, a, p) # equivalent to (g ** a) % p

# Bob's private key
b = 15
# Bob's public key
B = pow(g, b, p) # equivalent to (g ** b) % p

# Shared secret computation:
shared_alice = pow(B, a, p)
shared_bob = pow(A, b, p)
```

Is DH secure against a <u>passive</u> attacker?

> **NOTE**: A passive attacker can only eavesdrop on the communication.

Is DH secure against a <u>passive</u> attacker?

> **NOTE**: A passive attacker can only eavesdrop on the communication.

Passive attacker:

- from eavesdropping knows $g, p, g^a \bmod p, g^b \bmod p$
- can compute $g^{a+b} \bmod p$ which is different from $g^{ab} \bmod p$

Is DH secure against a <u>passive</u> attacker?

> **NOTE**: A passive attacker can only eavesdrop on the communication.

Passive attacker:

- from eavesdropping knows $g, p, g^a \bmod p, g^b \bmod p$
- can compute $g^{a+b} \bmod p$ which is different from $g^{ab} \bmod p$

Why it cannot compute the shared secret key?

Is DH secure against a <u>passive</u> attacker?

> **NOTE**: A passive attacker can only eavesdrop on the communication.

Passive attacker:

- from eavesdropping knows $g, p, g^a \bmod p, g^b \bmod p$
- can compute $g^{a+b} \bmod p$ which is different from $g^{ab} \bmod p$

Why it cannot compute the shared secret key?

To build the shared secret key it needs to recover a (or b). One way of breaking DH is to solve DLP, which is believed to be hard to break. However, there could be other ways of breaking DH that do not necessarily need to solve DLP. However, after several years it is still believed to be safe.

Is DH secure against an <u>active</u> attacker?

> **NOTE**: An active attacker can both eavesdrop and modify the communication.

Is DH secure against an <u>active</u> attacker?

> **NOTE**: An active attacker can both eavesdrop and modify the communication.

Man-in-the-Middle (MITM) attack:

1. Adversary fakes Bob identity and performs DH with Alice.
2. Adversary fakes Alice identity and performs DH with Bob.
3. Alice and Bob think that they have a common shared key, instead they are sharing a secret key with the adversary.

Is DH secure against an <u>active</u> attacker?

NOTE: An active attacker can both eavesdrop and modify the communication.

Man-in-the-Middle (MITM) attack:

1. Adversary fakes Bob identity and performs DH with Alice.
2. Adversary fakes Alice identity and performs DH with Bob.
3. Alice and Bob think that they have a common shared key, instead they are sharing a secret key with the adversary.

PROBLEM:
Alice and Bob do not ***authenticate*** the received public keys => the attacker is able to impersonate both parties.

RSA (Rivest–Shamir–Adleman) cryptosystem

Solving the Key Management Problem

In a network with n users, using symmetric encryption requires **one shared key for each pair of users**.

- Leads to having to manage $n(n-1)/2$ keys.
- E.g. 10 users → 45 keys; 100 users → 4,950 keys!

Solving the Key Management Problem

In a network with n users, using symmetric encryption requires **one shared key for each pair of users**.

- Leads to having to manage $n(n-1)/2$ keys.
- E.g. 10 users → 45 keys; 100 users → 4,950 keys!

Asymmetric Cryptography tackles this issue by:

- Giving each user a **single key pair**: a public key (shared) and a private key (kept secret).
- Eliminating the need to pre-share secret keys between every pair of users.
- Allowing anyone to send encrypted messages to a user using their public key, which only the owner can decrypt with its private key.

RSA Scheme

Variables:

- $n = p \cdot q$ - modulus
- e - public exponent
- d - private exponent
- m - plaintext message
- c - ciphertext

Keys:

- Public Key: (n, e)
- Private Key: (n, d)

Formulas:

- **Encryption:** $c \equiv m^e \pmod{n}$
- **Decryption:** $m \equiv c^d \pmod{n}$

RSA: Is it secure?

> The main assumption behind RSA is that, given (e, n), it should be hard to compute d, p, q or $\Phi(n)$.

To make this hard:

- p and q should be two large (more than 1024 bits) distinct prime numbers.
- p and q shouldn't be close.
- e should be equal or larger than 3.
- ...

RSA: Is it secure?

The main assumption behind RSA is that, given (e, n), it should be hard to compute d, p, q or $\Phi(n)$.

To make this hard:

- p and q should be two large (more than 1024 bits) distinct prime numbers.
- p and q shouldn't be close.
- e should be equal or larger than 3.
- ...

WARNING: even when recommendations are followed, RSA suffers from different problems.

RSA: what if two moduli share a common factor p?

$$N_1 = p \cdot q_1$$
$$N_2 = p \cdot q_2$$
$$gcd(N_1, N_2) = p$$

Then we can easily recover q_1 and q_2:

$$q_1 = N_1/p$$
$$q_2 = N_2/p$$

NOTE: knowing N,p,q we can compute $\phi(N)$ and recover d.

RSA: small message, small exponent e

If x and e are small then the modulo operation does nothing:

$$y = x^e \, mod \, n = x^e$$

Adversary can easily get the plaintext, without factoring n by computing:

$$x = \sqrt[e]{y}$$

RSA: small message, small exponent e

If x and e are small then the modulo operation does nothing:

$$y = x^e \, mod \, n = x^e$$

Adversary can easily get the plaintext, without factoring n by computing:

$$x = \sqrt[e]{y}$$

Corner case: if $e = -1$ then

$$y \equiv x^e \equiv x^{-1} \pmod{N}$$

The ciphertext is simply the modular inverse of the plaintext.

Any adversary that knows N can immediately recover the plaintext by computing:

$$x \equiv y^{-1} \pmod{N}$$

Digital Signatures: OpenGPG

Beyond Confidentiality

Cryptography is not only about keeping messages secret (confidentiality). It is also used to provide:

- **Integrity:** Ensure the message has not been altered.
- **Authentication:** Verify the identity of the sender.
- **Non-repudiation:** Prevent the sender from denying that they sent the message.

Beyond Confidentiality

Cryptography is not only about keeping messages secret (confidentiality). It is also used to provide:

- **Integrity:** Ensure the message has not been altered.
- **Authentication:** Verify the identity of the sender.
- **Non-repudiation:** Prevent the sender from denying that they sent the message.

> These goals can be achieved through **Digital Signatures**.

Digital signatures work in the **public-key cryptography** setting:

- The sender **signs** a message using their **private key**.
- Anyone can **verify** the signature using the sender's **public key**.

PGP: Pretty Good Privacy

PGP

- Example of **digitals signatures** in practice.
- Software created for email and file encryption.
- Enables users to communicate securely using public-key cryptography.

PGP: Pretty Good Privacy

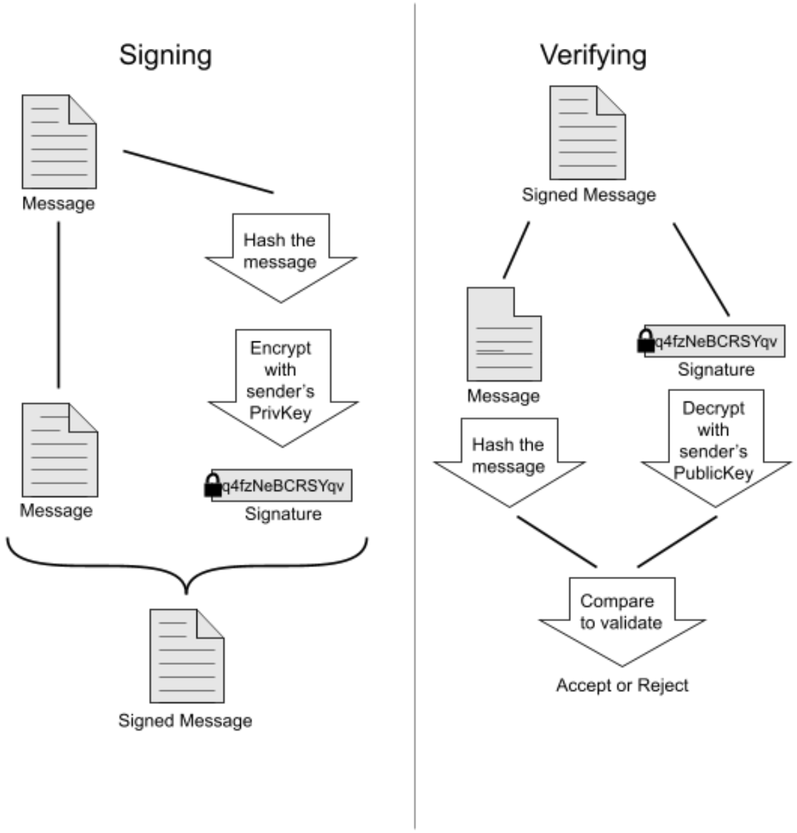
PGP

- Example of **digitals signatures** in practice.
- Software created for email and file encryption.
- Enables users to communicate securely using public-key cryptography.

OpenPGP

- Standard derived from PGP that defines format and rules for encrypted messages, digital signatures, and key exchange between systems.
- Enables interoperability between different PGP-compatible programs and promotes open encryption standards.

OpenPGP: Signing & Verification



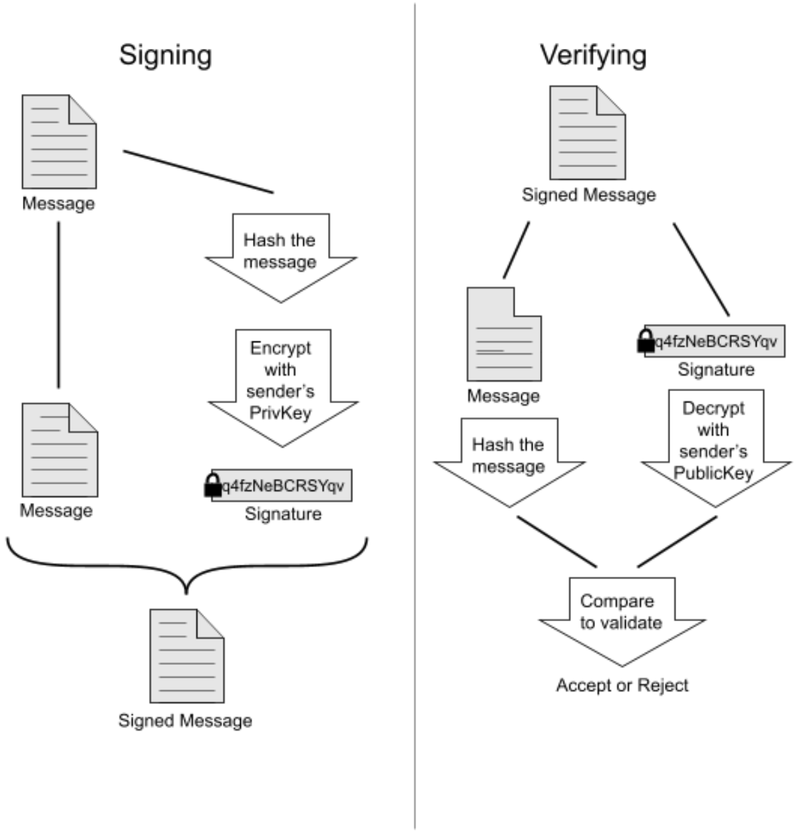
Signing (Sender Side)

- The original file is processed through a hash function.
- The hash is encrypted with the sender's private key.
- The result is the digital signature, attached to the file.

Verification (Receiver Side)

- The receiver computes the hash of the received file.
- The signature is decrypted using the sender's public key.
- The two hashes are compared.

OpenPGP: Signing & Verification



Signing (Sender Side)

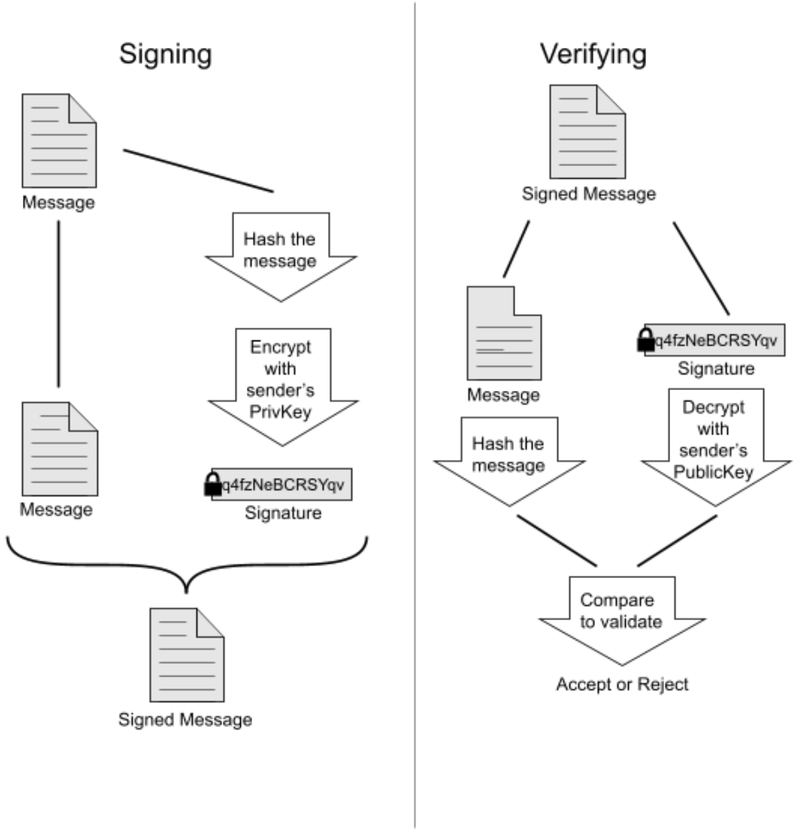
- The original file is processed through a hash function.
- The hash is encrypted with the sender's private key.
- The result is the digital signature, attached to the file.

Verification (Receiver Side)

- The receiver computes the hash of the received file.
- The signature is decrypted using the sender's public key.
- The two hashes are compared.

If they match → the file is authentic and unmodified.

OpenPGP: Signing & Verification



Signing (Sender Side)

- The original file is processed through a hash function.
- The hash is encrypted with the sender's private key.
- The result is the digital signature, attached to the file.

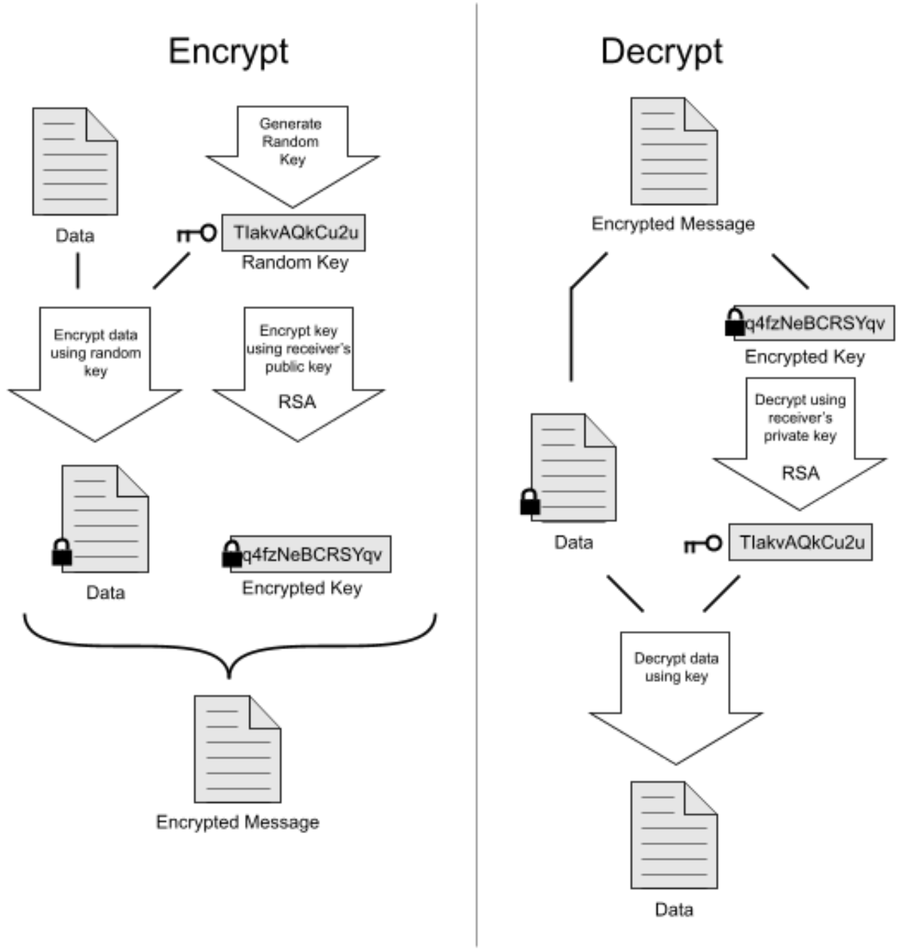
Verification (Receiver Side)

- The receiver computes the hash of the received file.
- The signature is decrypted using the sender's public key.
- The two hashes are compared.

If they match → the file is authentic and unmodified.

If they differ → the file was altered or the signature is invalid.

OpenPGP: Encryption & Decryption



Encryption (Sender Side)

- A random symmetric key is generated.
- The file is encrypted using the symmetric key.
- The symmetric key is encrypted with the receiver's public key (e.g., RSA).
- The encrypted file + encrypted key are sent together.

Decryption (Receiver Side)

- The encrypted symmetric key is decrypted using the receiver's private key.
- The recovered symmetric key is used to decrypt the file.

OpenPGP in practice

GPG

- Most popular implementation of the OpenPGP standard.

Allows users to:

- Generate public/private key pairs.
- Export and import keys.
- Encrypt and decrypt files.
- Create and verify digital signatures.
- ...

Playing with OpenPGP

Windows

- Tool: https://www.gpg4win.org/
- After a short installation wizard, **Kleopatra** is installed on the system, providing a graphical interface for managing keys, encryption, and signatures.

MacOS

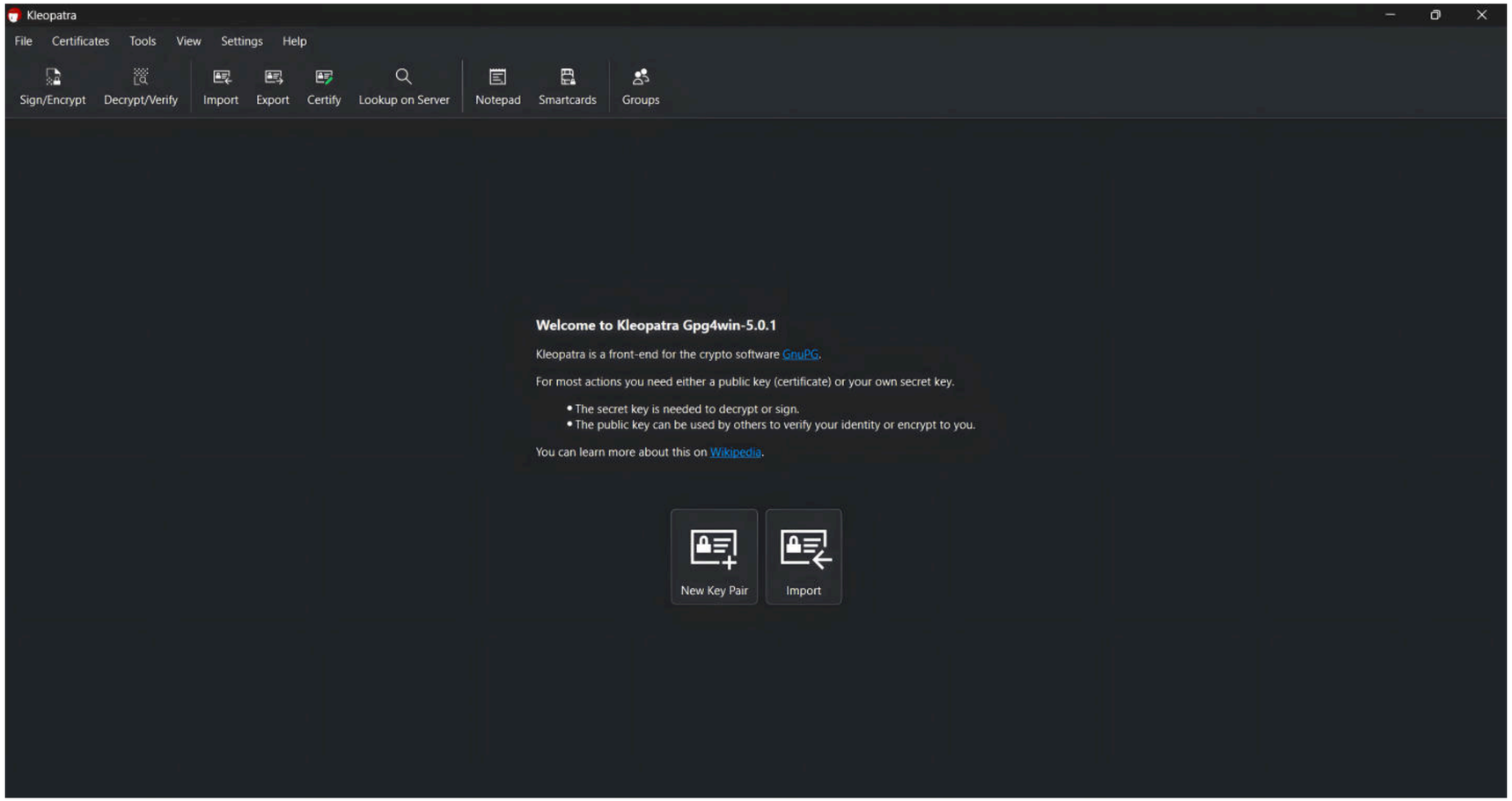
- Tool: https://gpgtools.org/
- After installation, **GPG Suite** (**GPGTools**) is available on the system, integrating OpenPGP functionality into macOS apps and providing key management tools.

gpg4win

Windows Tool

gpg4win: interface overview

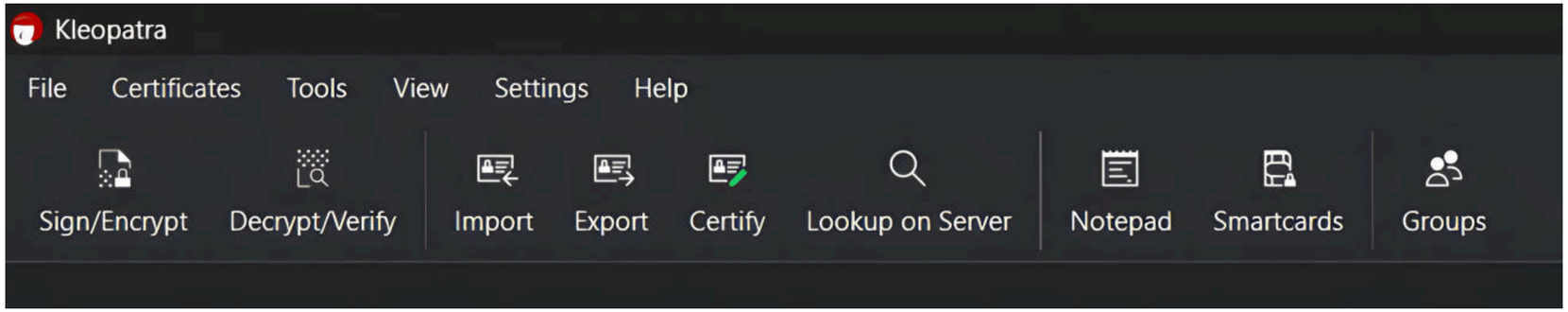
Initial page of the Kleopatra interface:



gpg4win: action toolbar

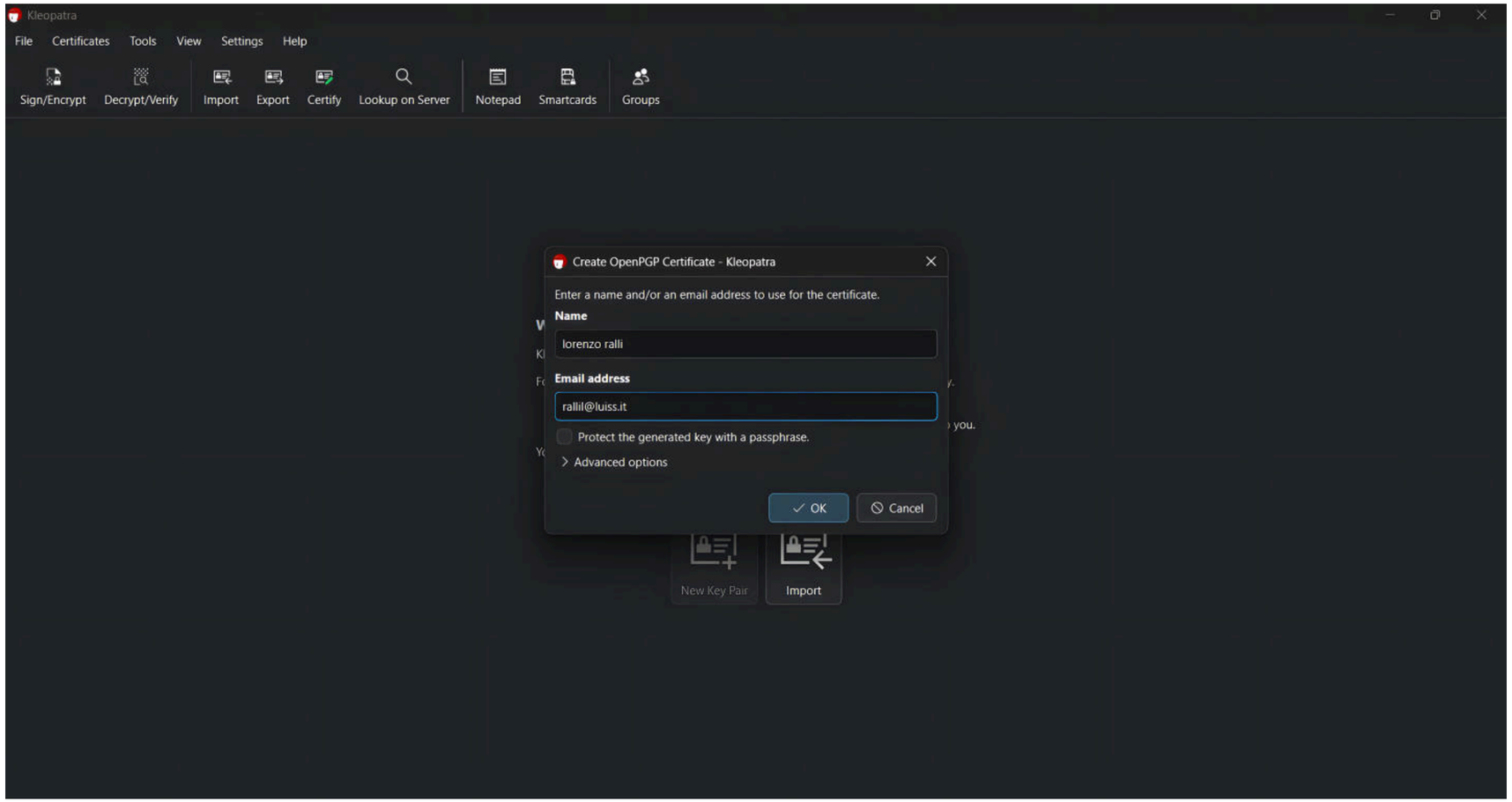
Offer quick access to:

- Signing.
- Encryption.
- Verification.
- Decryption.
- Import and export of keys.
- ...



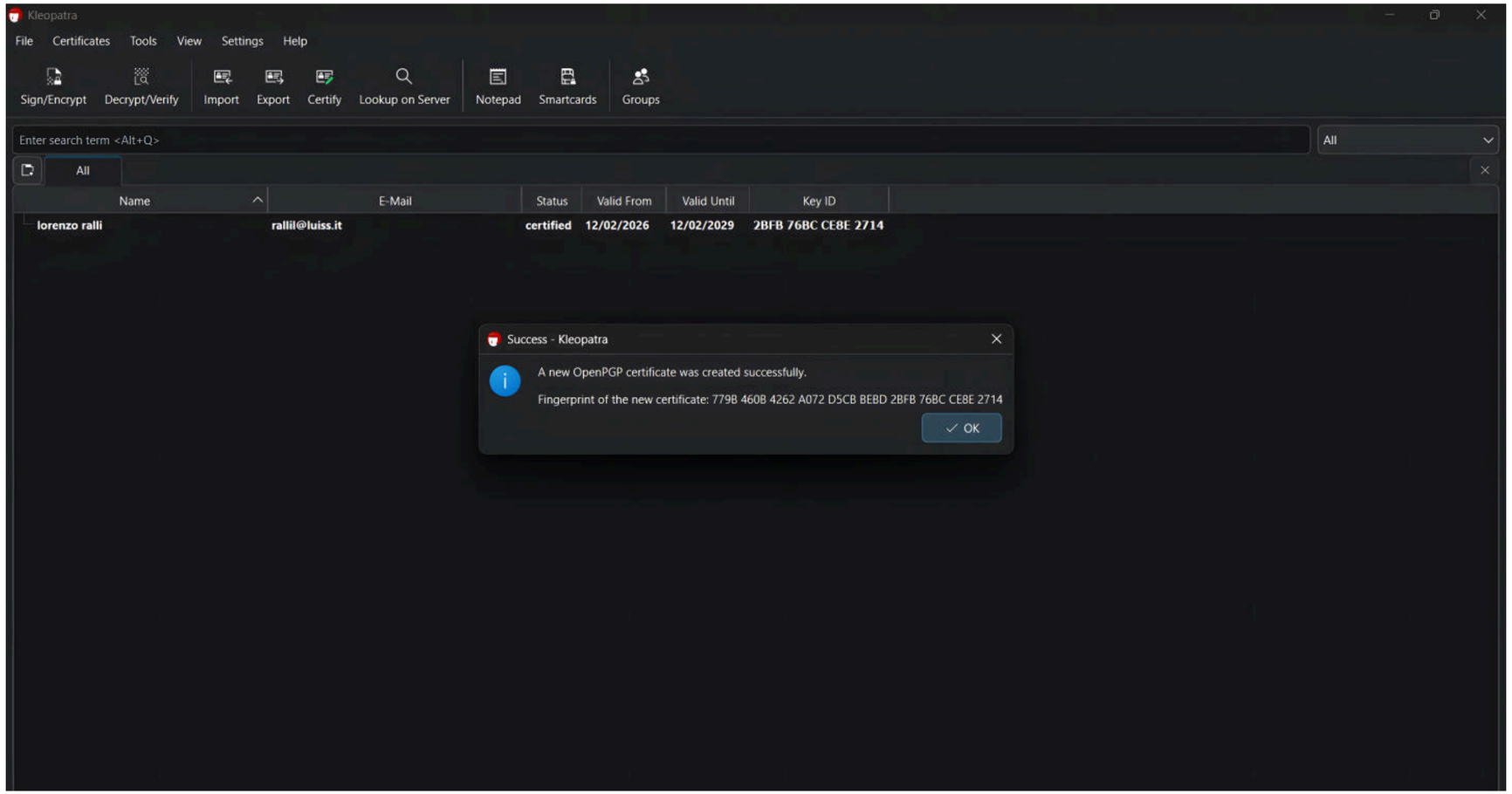
gpg4win: creating a new key pair

Click *New Key Pair* → Enter Name & Email → Generate Keys:



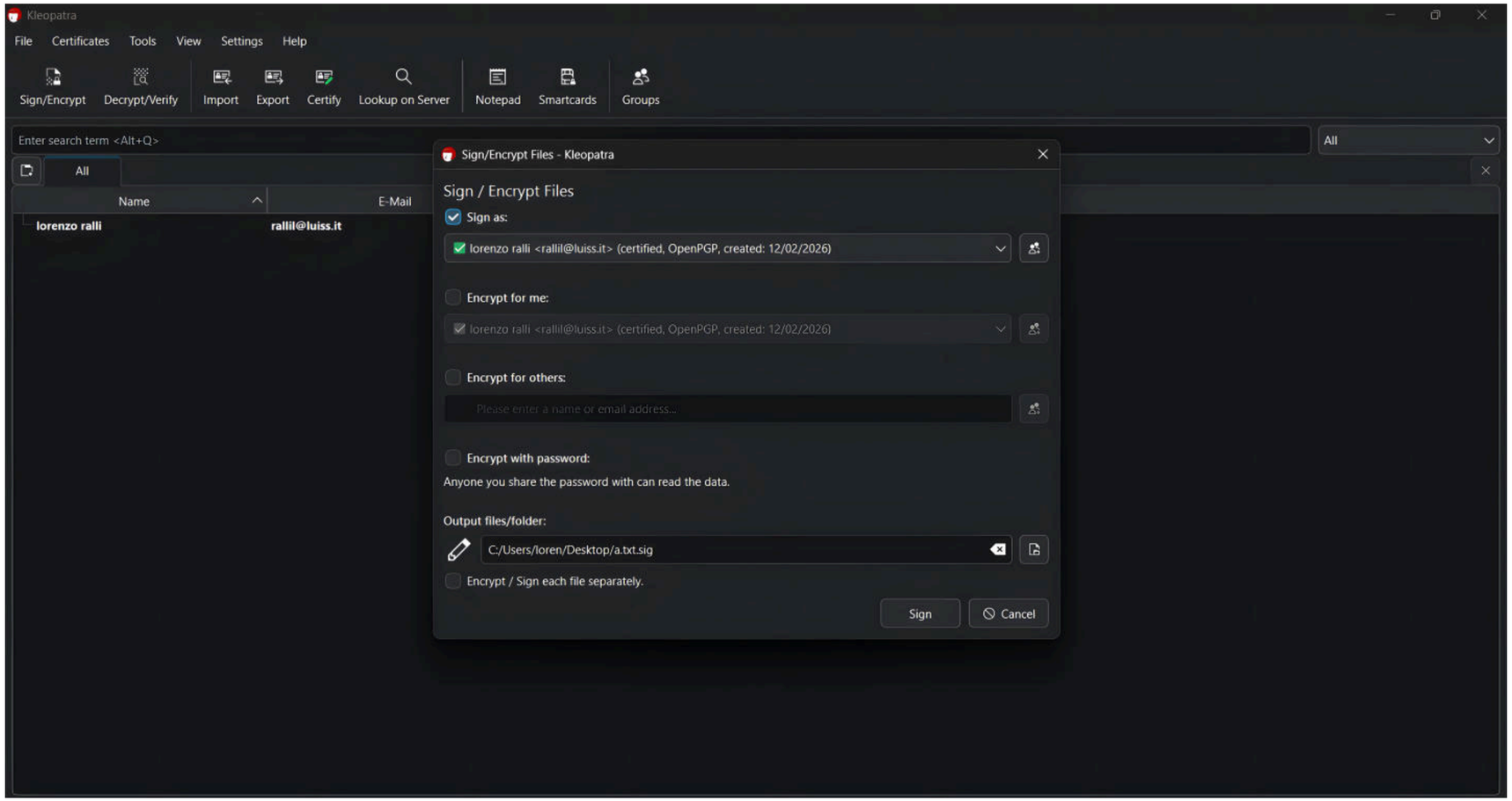
gpg4win: creating a new key pair

Success screen:



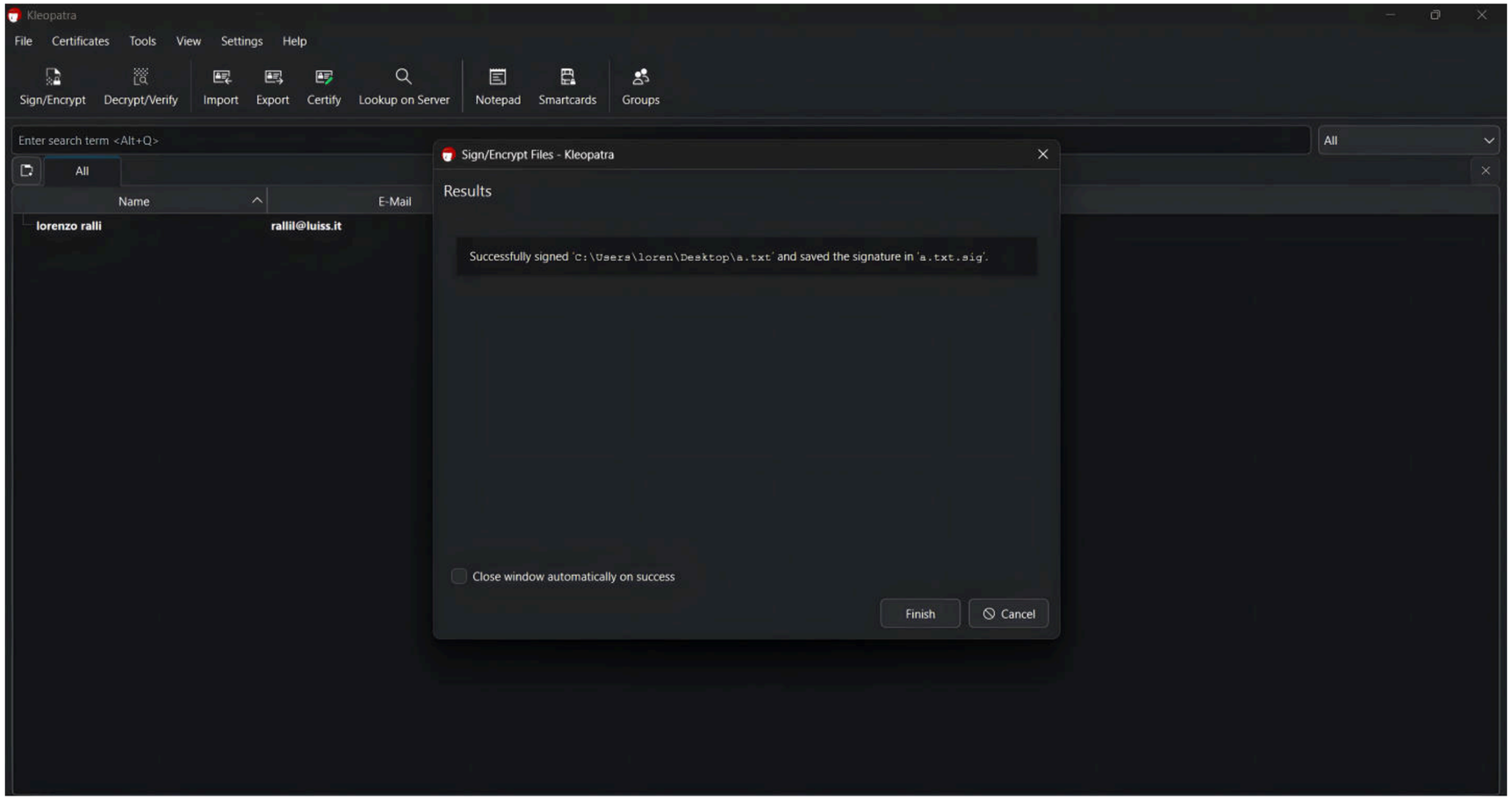
gpg4win: signing a file

Click *Sign/Encrypt* in the action toolbar → choose your signing key:



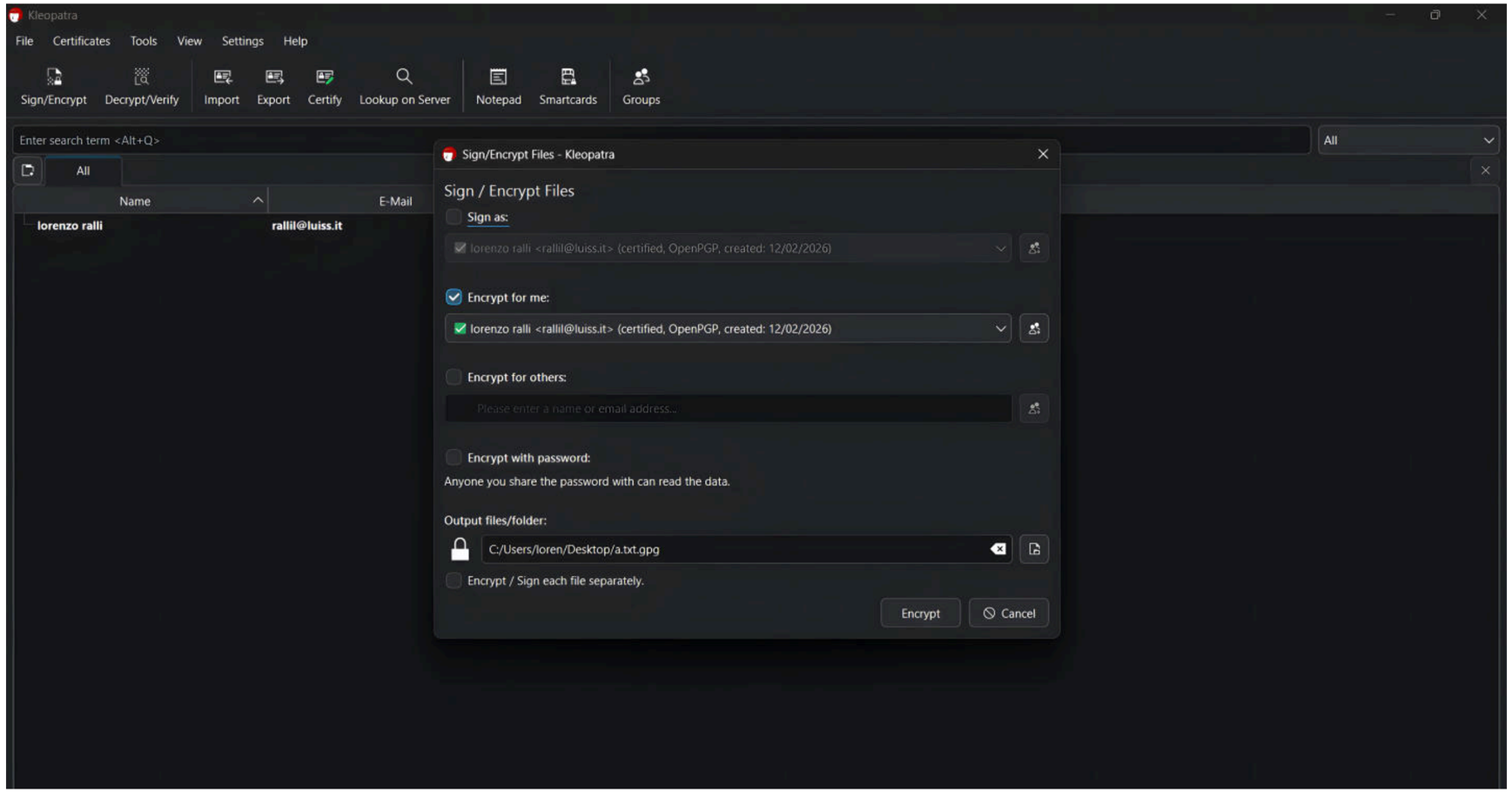
gpg4win: signing a file

Success screen:



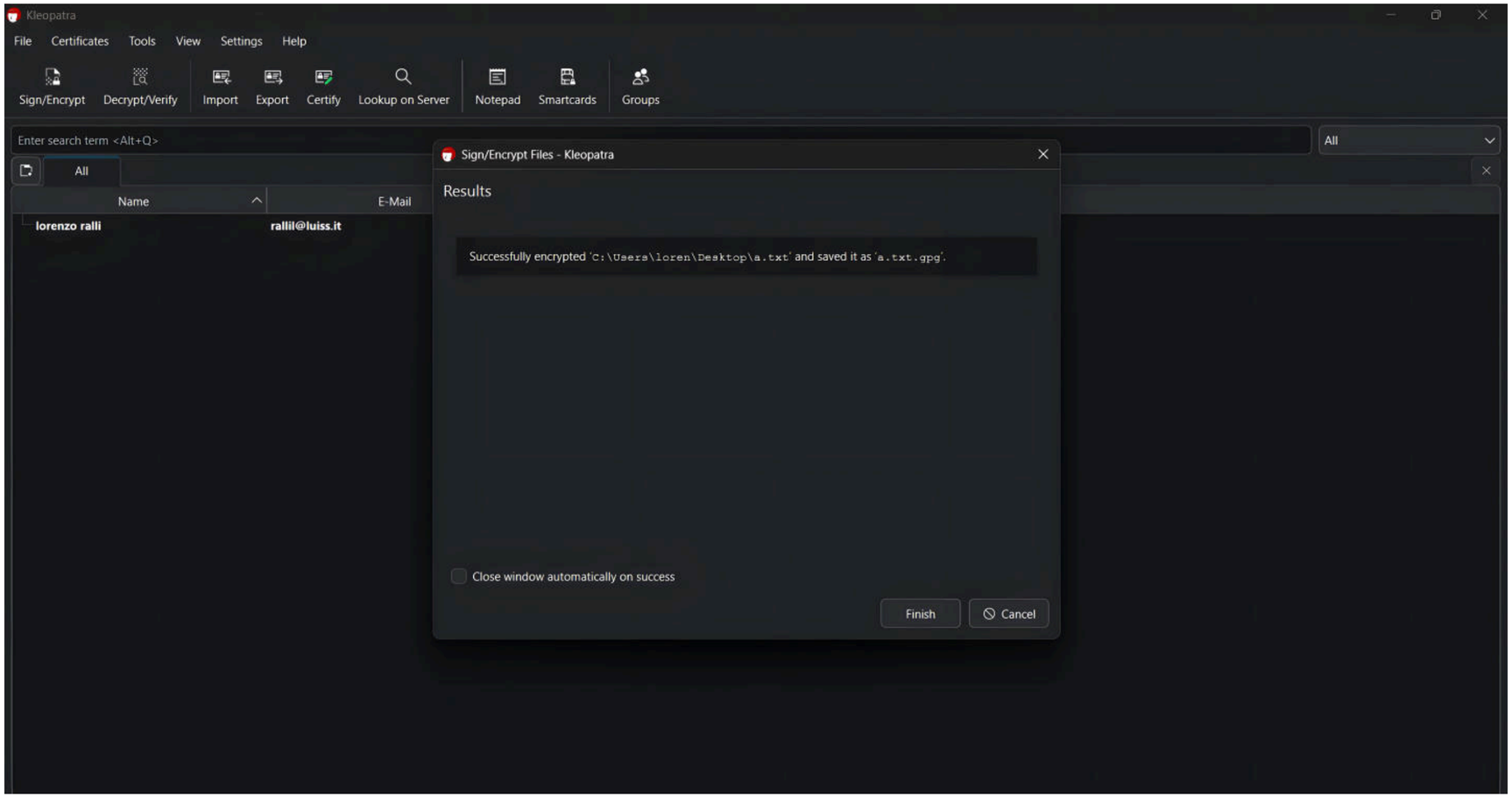
gpg4win: encrypting a file

Click *Sign/Encrypt* in the action toolbar → choose your encryption key:



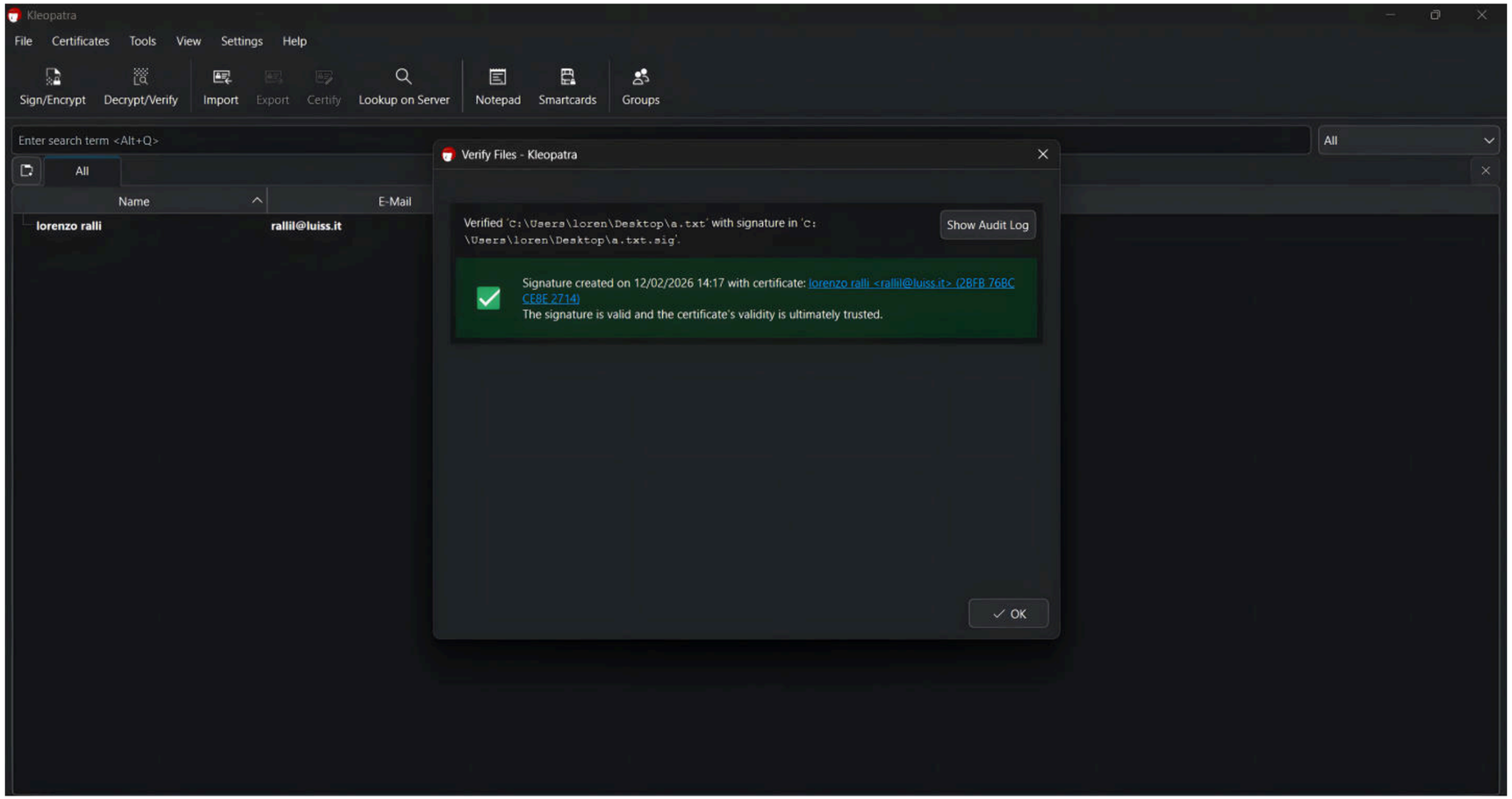
gpg4win: encrypting a file

Success screen:



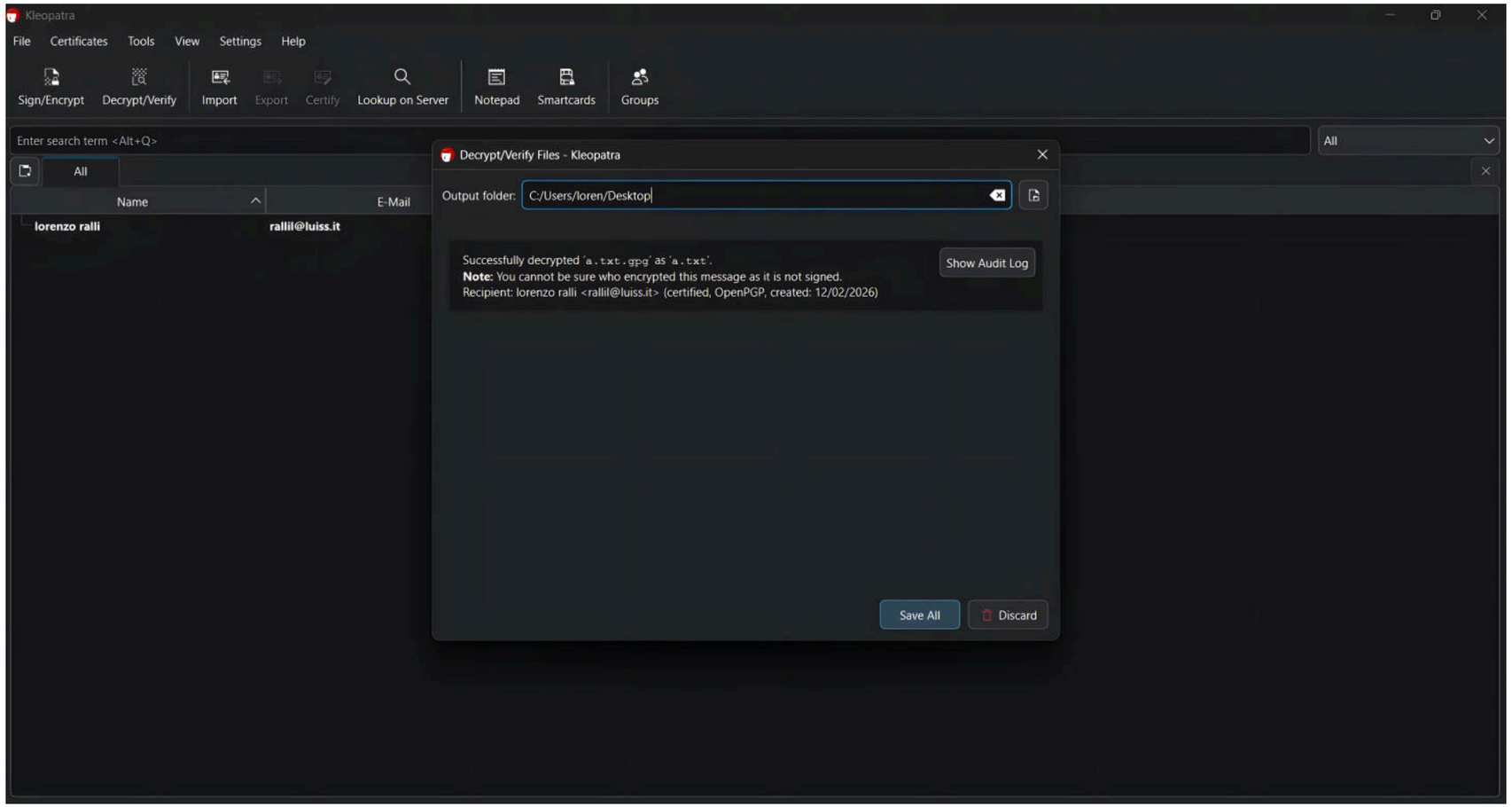
gpg4win: verifying a signature

Click *Decrypt/Verify* in the action toolbar → choose the signed file to verify in the *Open* window.



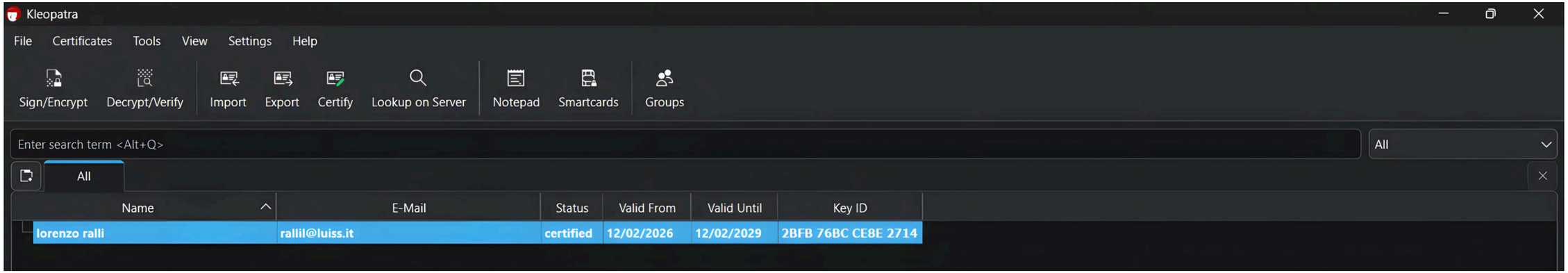
gpg4win: decrypting a file

Click *Decrypt/Verify* in the action toolbar → choose the encrypted file to decrypt in the *Open* window.



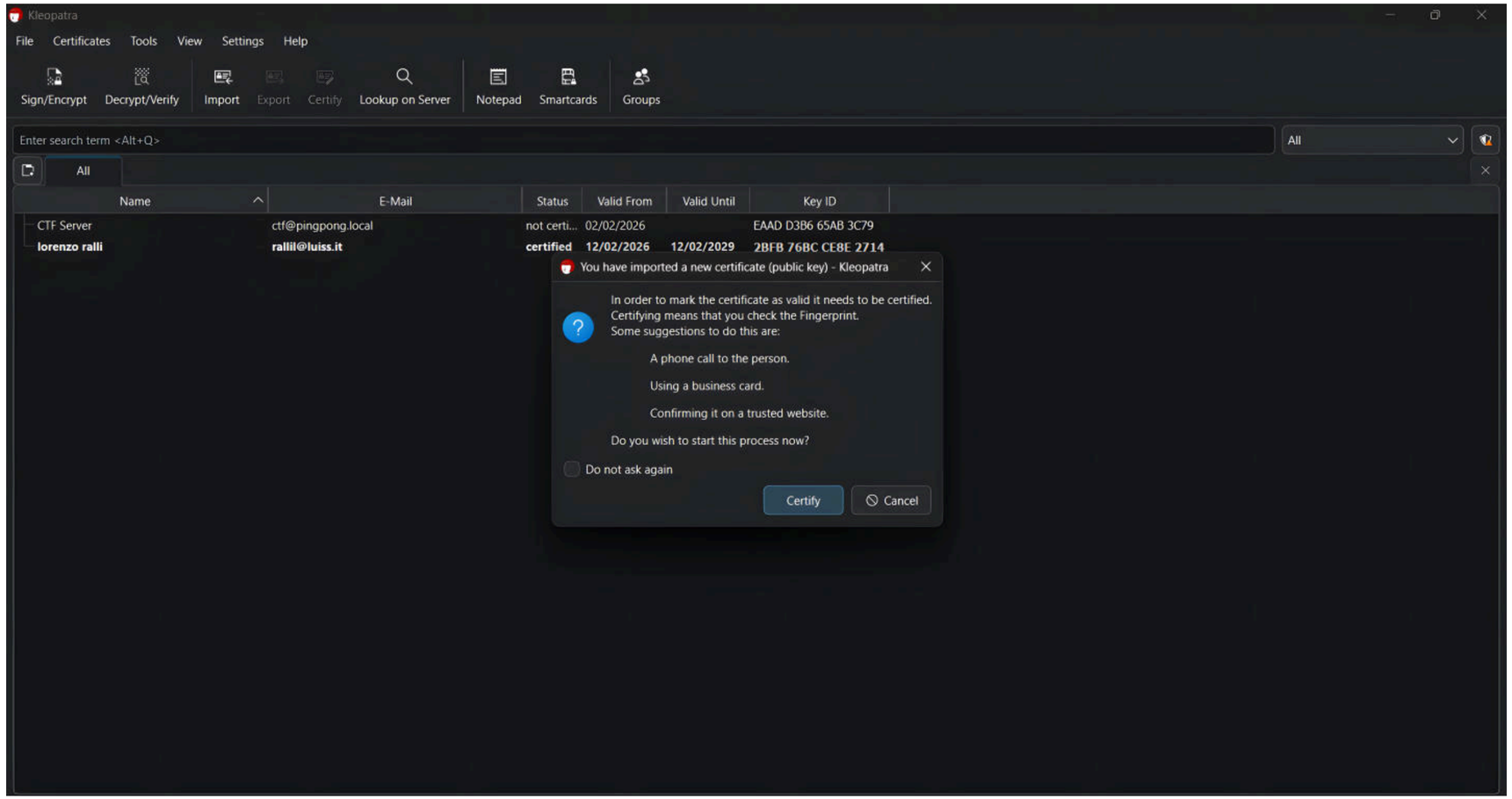
gpg4win: exporting public key

Select the key to export (by clicking on it) → click *Export* in the action toolbar → choose where to save it in the *Export* window:



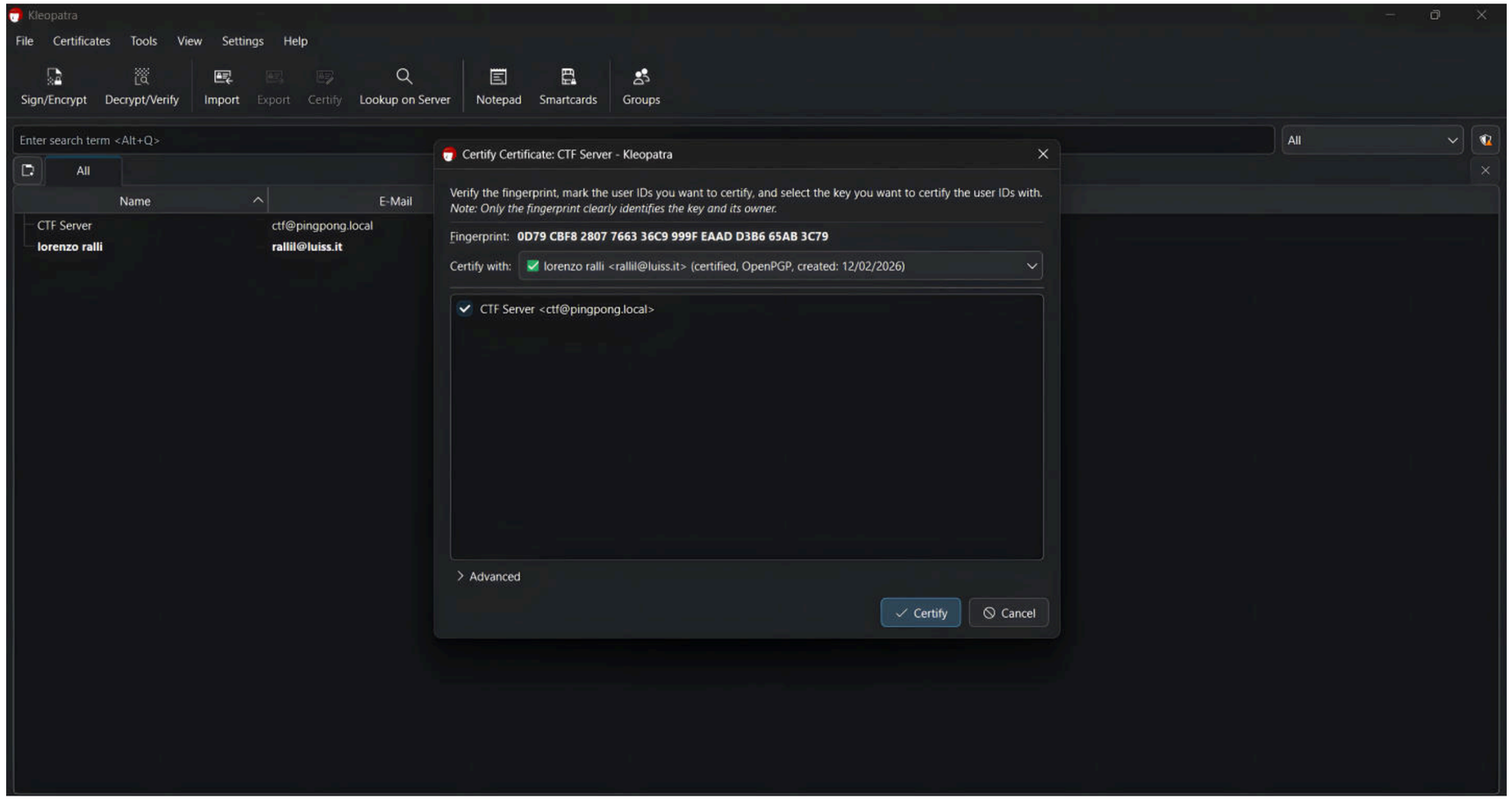
gpg4win: importing public key

Click *Import* in the action toolbar → select the key file → certify it by verifying its fingerprint:



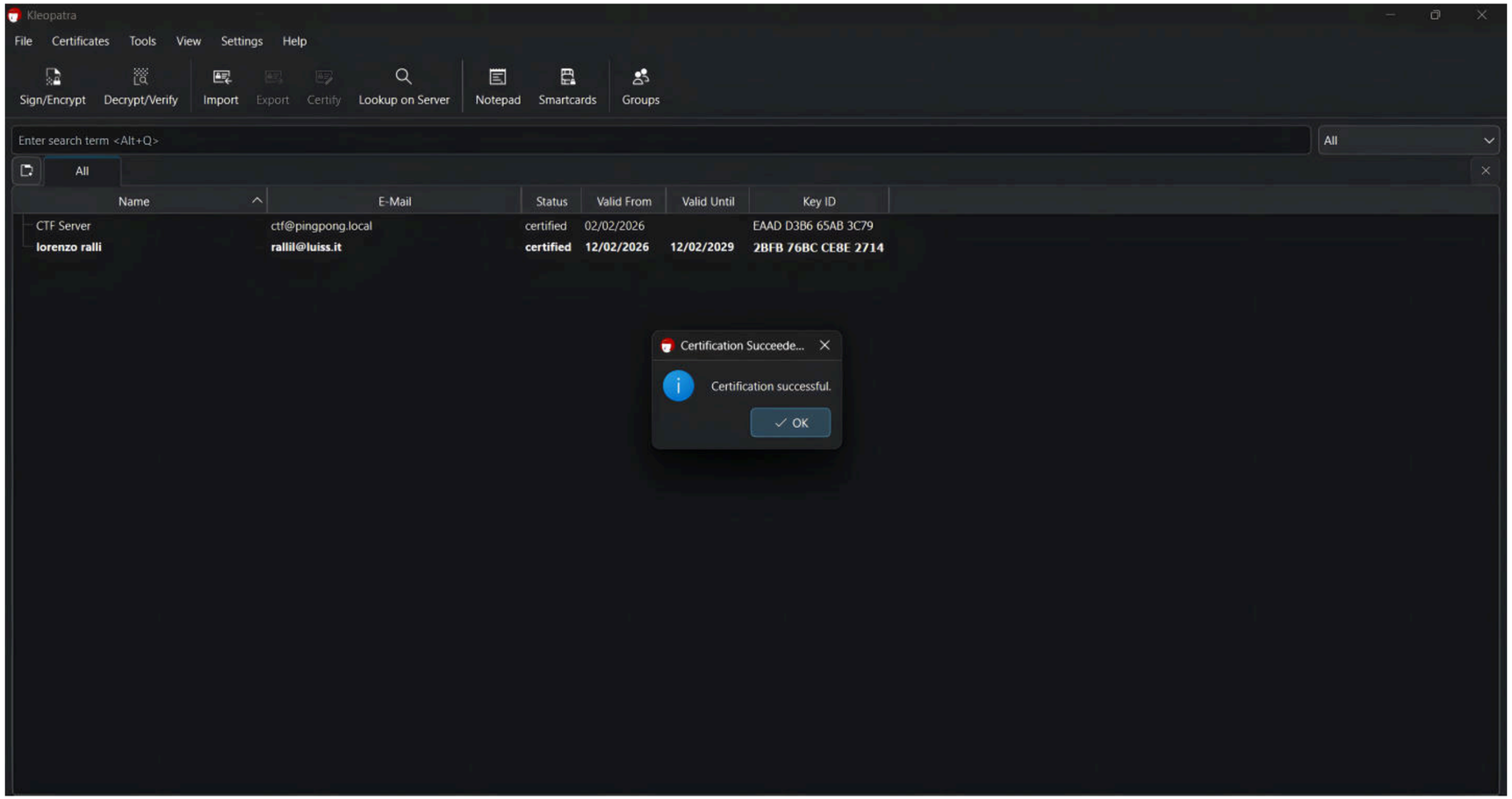
gpg4win: importing public key

Click *Import* in the action toolbar → select the key file → certify it by verifying its fingerprint:



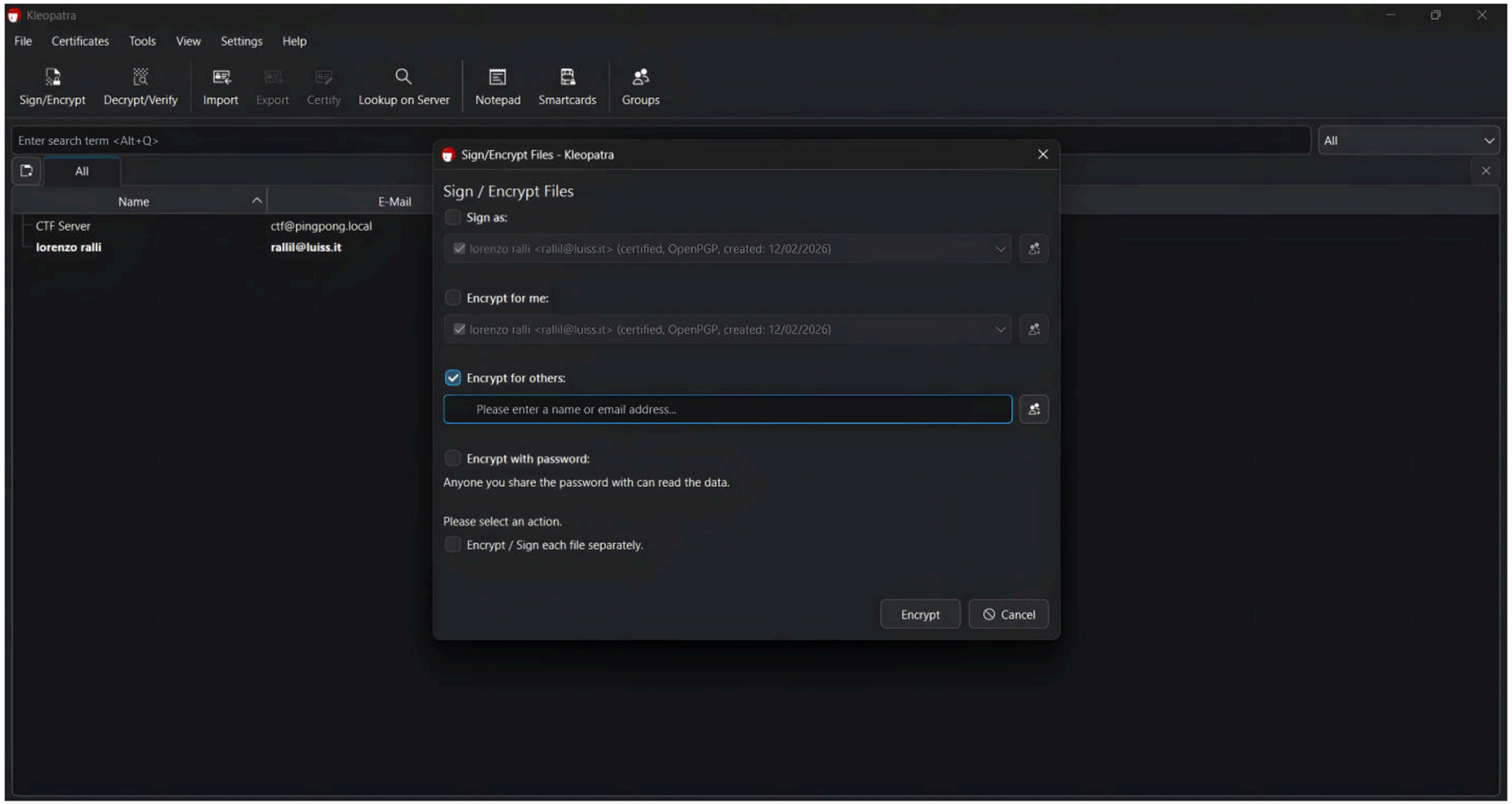
gpg4win: importing public key

Success screen:



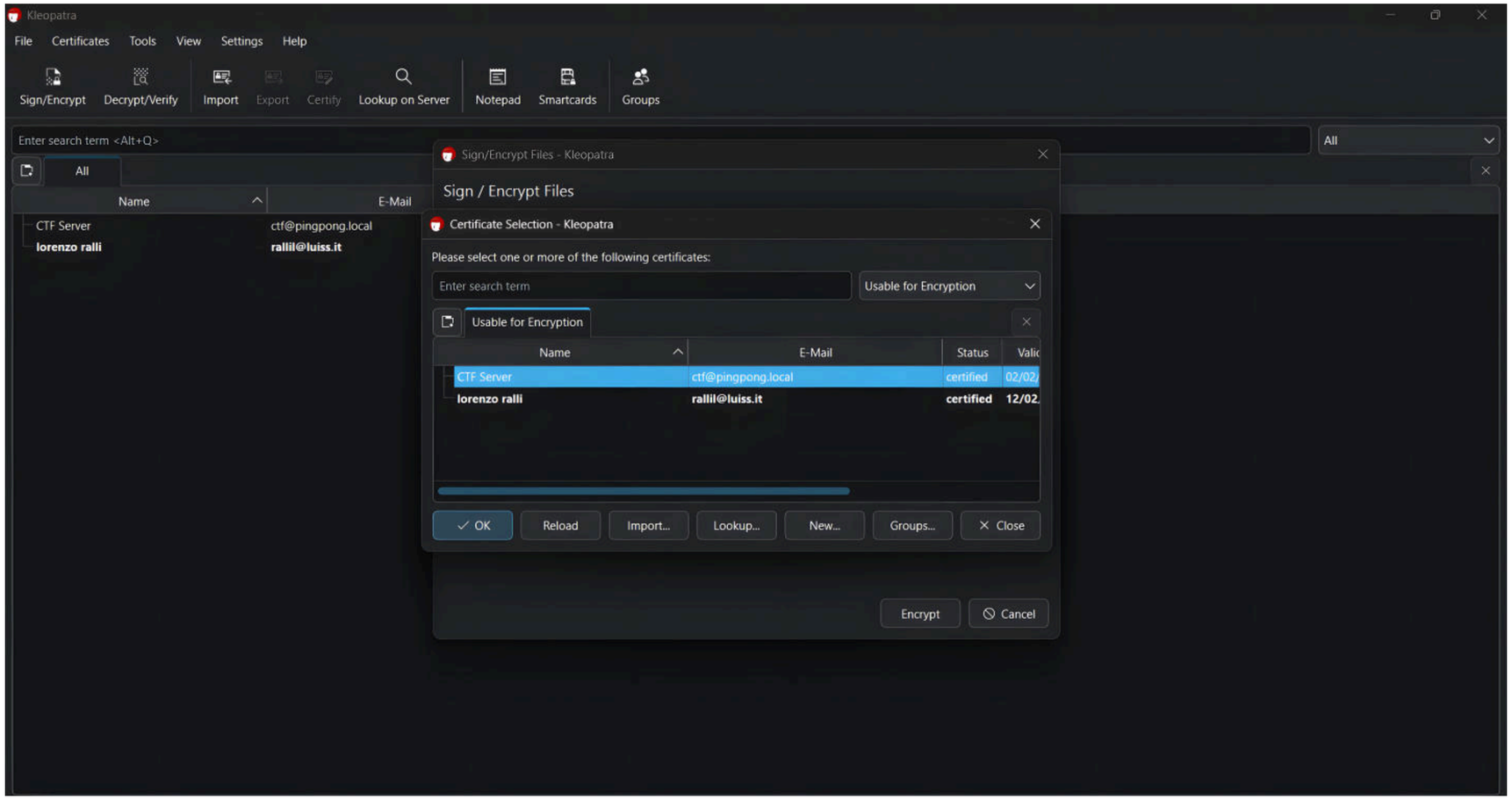
gpg4win: encrypting a file with the imported public key

Click *Sign/Encrypt* in the action toolbar → select *Encrypt for others* → pick the imported key:



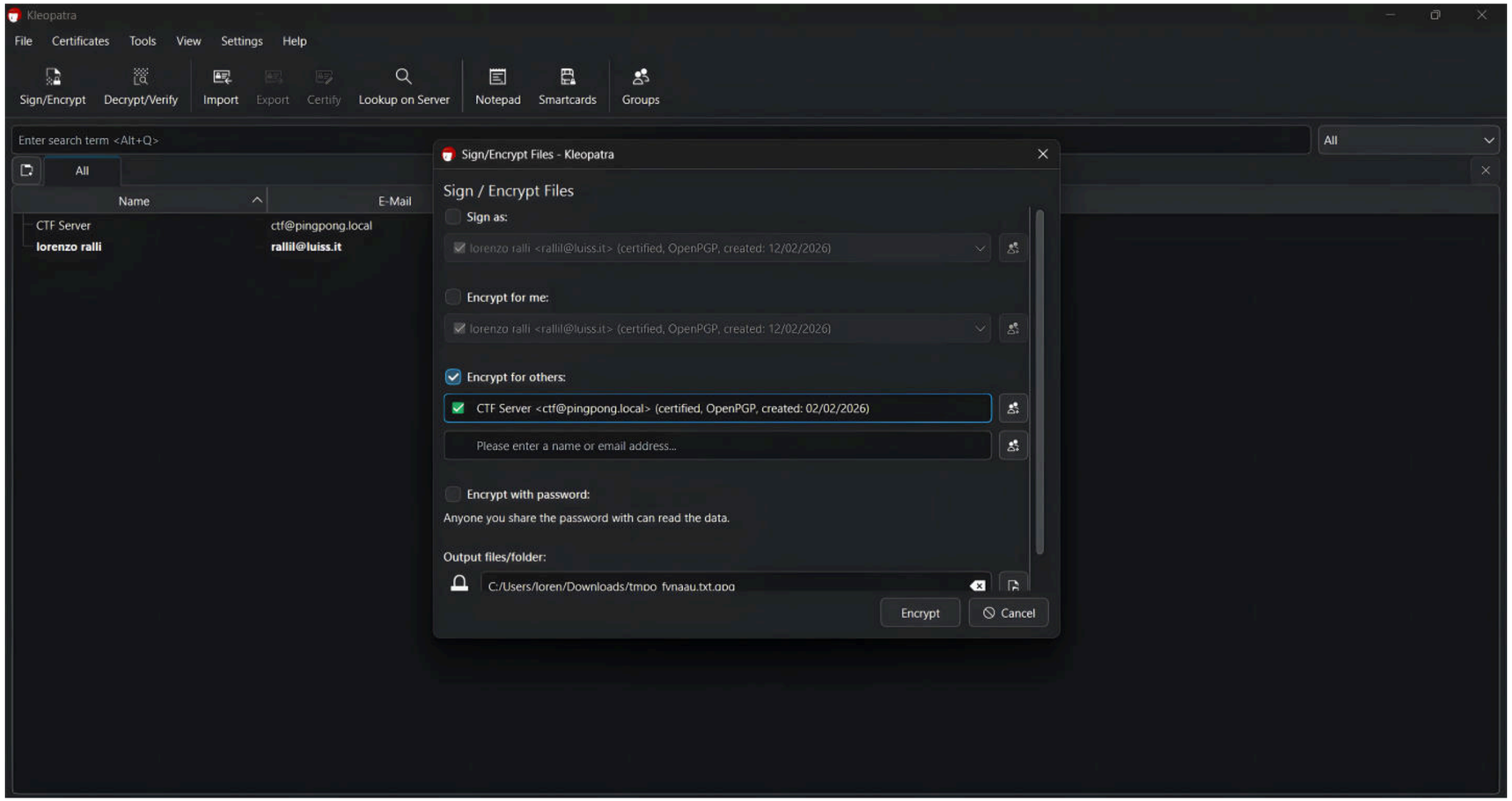
gpg4win: encrypting a file with the imported public key

Click *Sign/Encrypt* in the action toolbar → select *Encrypt for others* → pick the imported key:



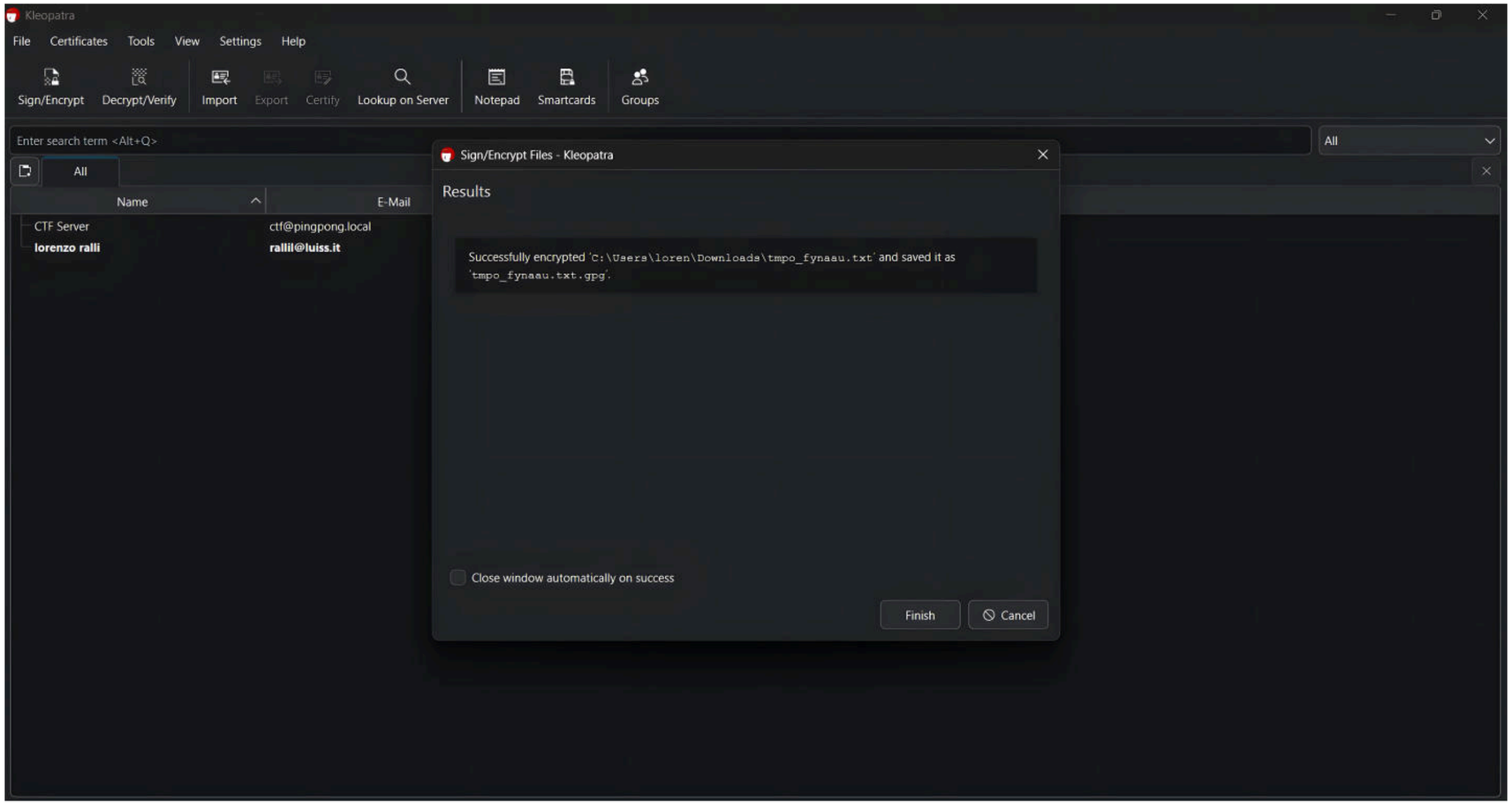
gpg4win: encrypting a file with the imported public key

Click *Sign/Encrypt* in the action toolbar → select *Encrypt for others* → pick the imported key:



gpg4win: encrypting a file with the imported public key

Success screen:

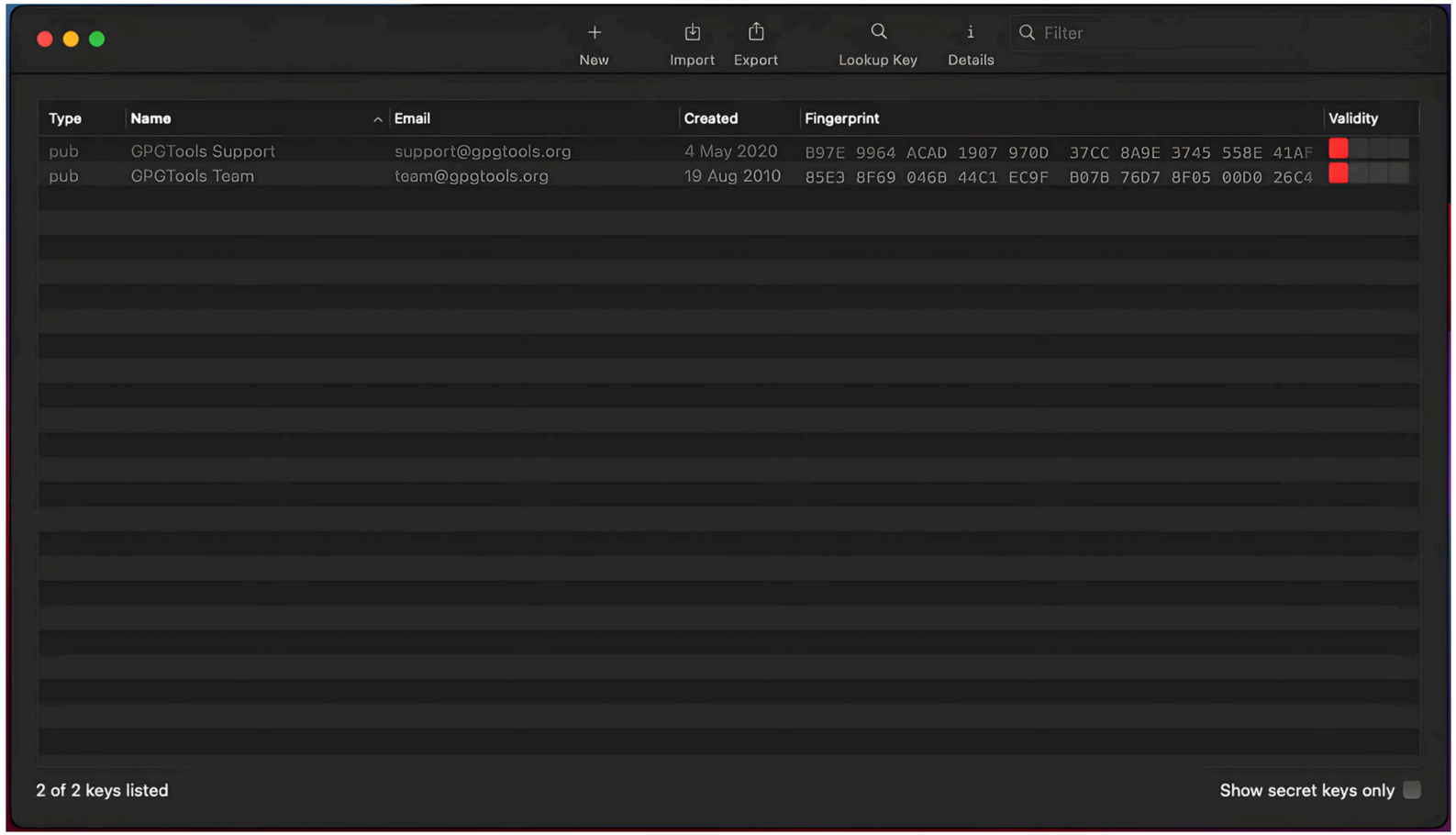


gpgtools

MacOS

gpgtools: interface overview

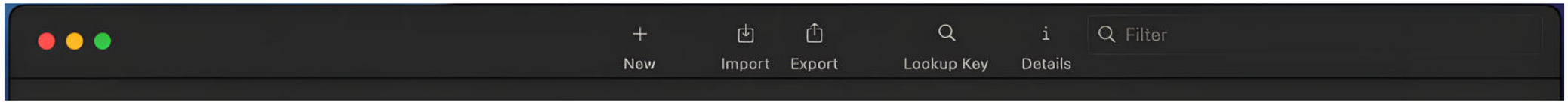
Initial page of gpgtools:



gpgtools: action toolbar and context menu

The action toolbar offers quick access to:

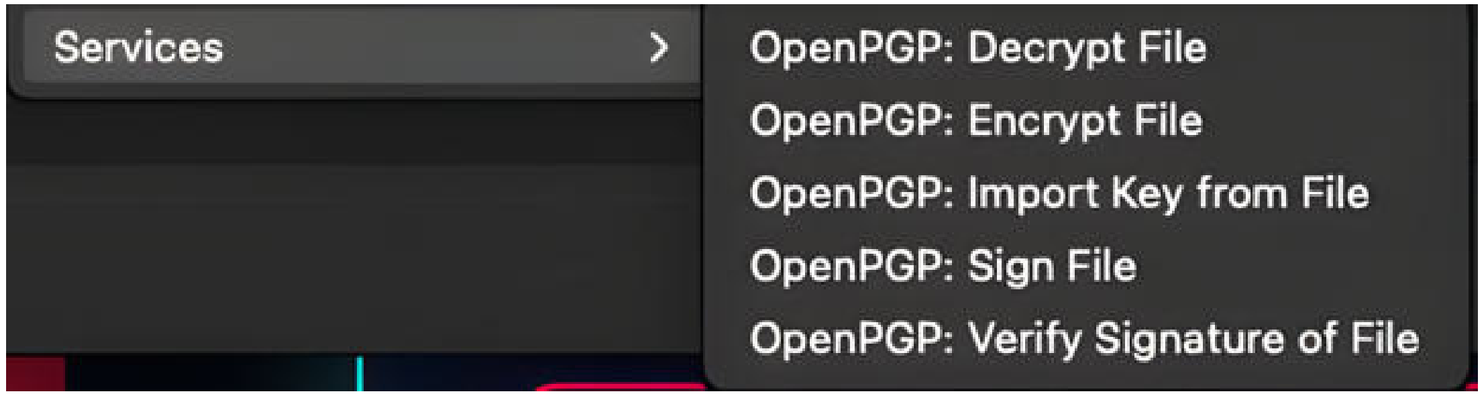
- Creating new key pair.
- Import and export of keys.



gpgtools: action toolbar and context menu

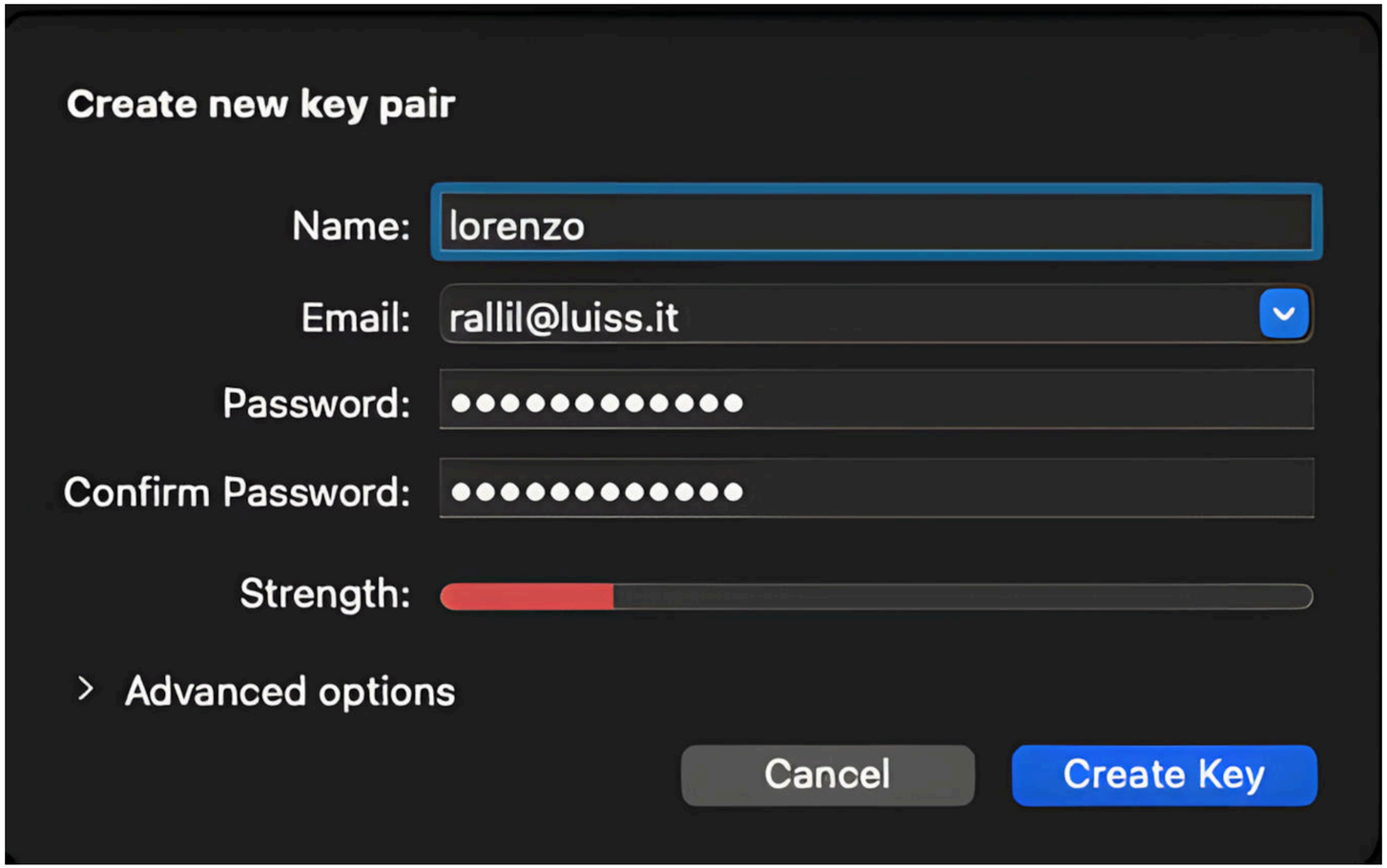
The context menu offers quick access to:

- Signing and Verification.
- Encryption and Decryption.
- Importing a public key from file.



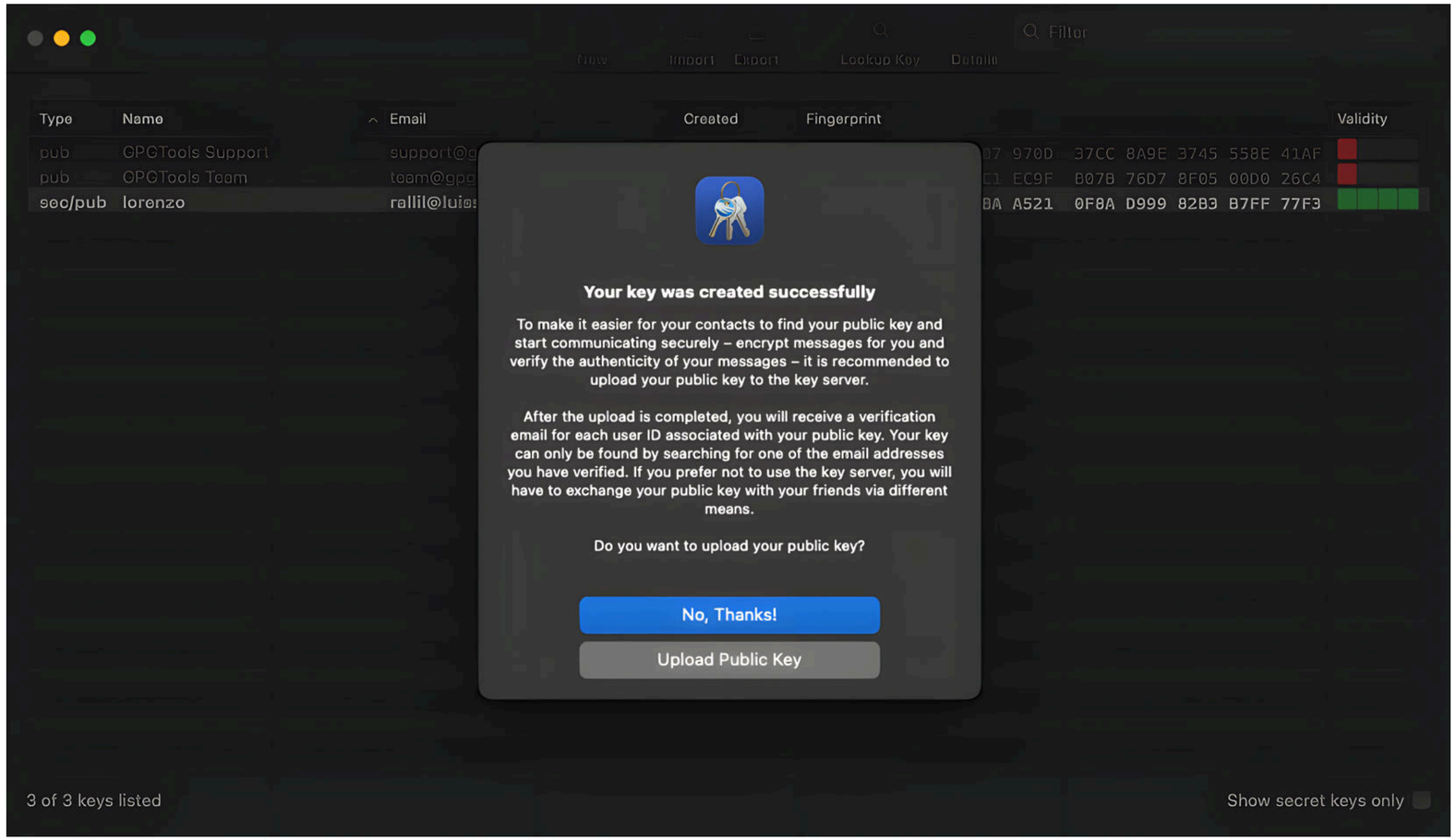
gpgtools: creating a new key pair

Click *New* in the action toolbar → fill the form:



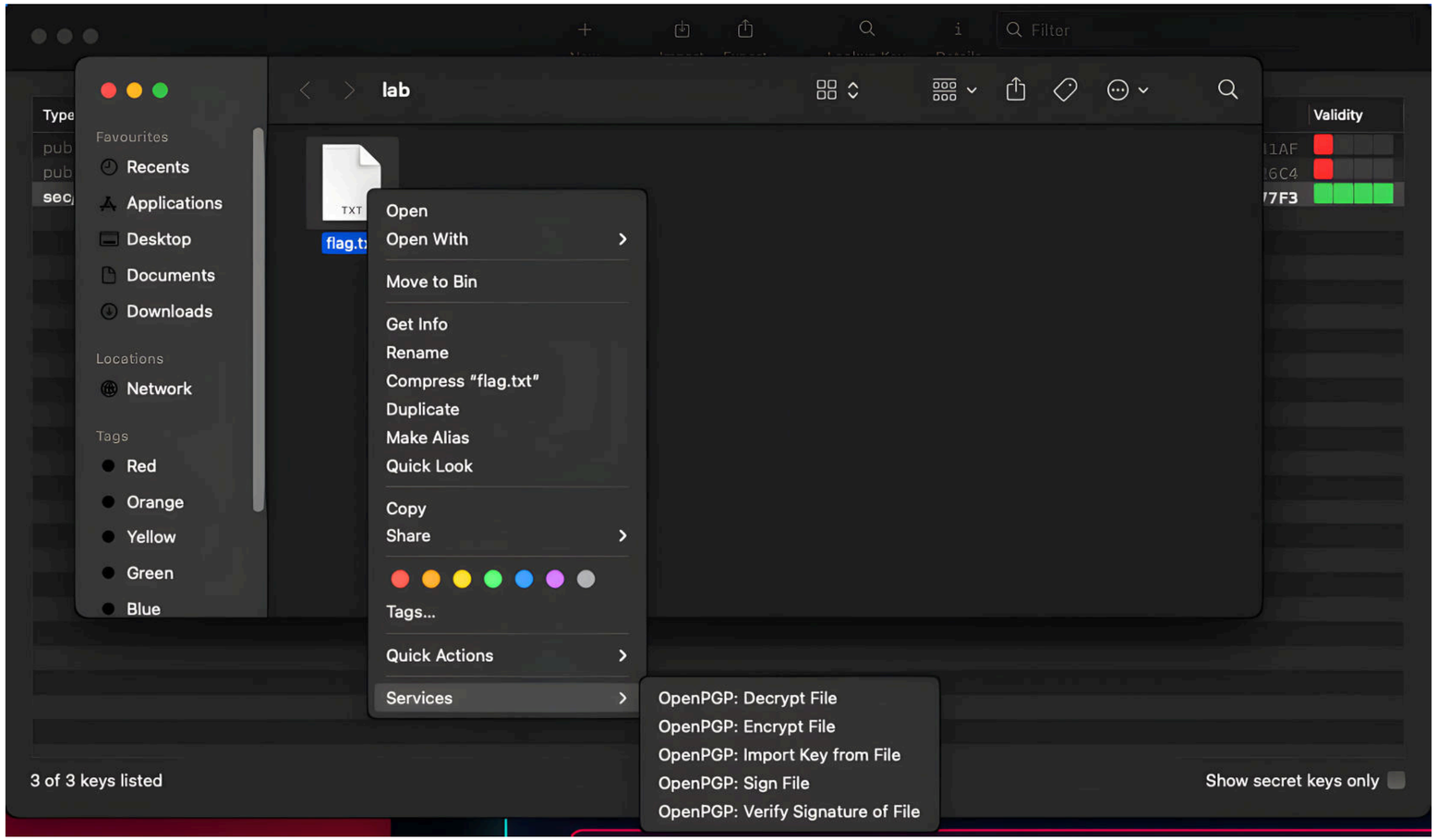
gpgtools: creating a new key pair

Success screen:



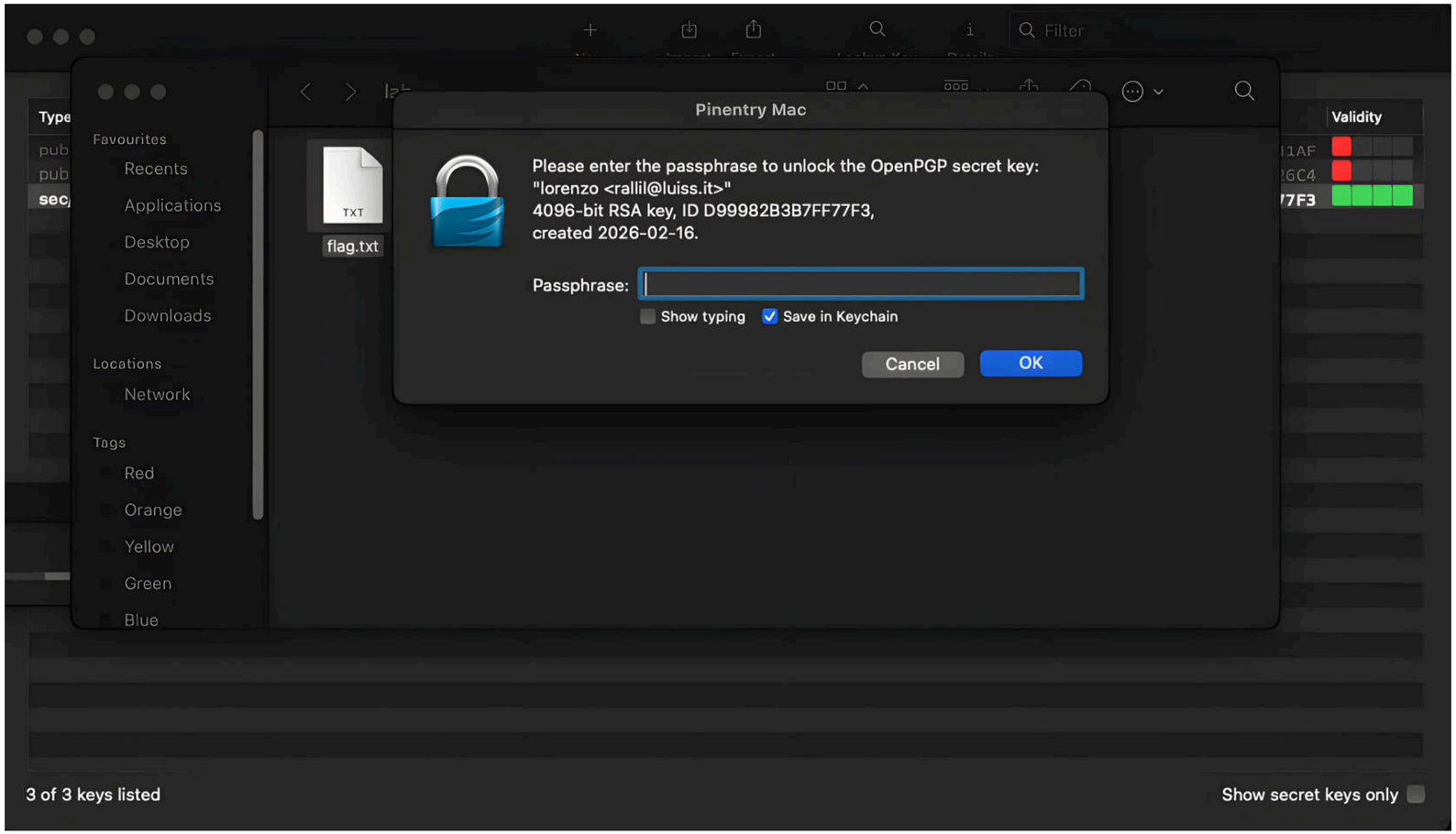
gpgtools: signing a file

Right click the file to sign to spawn the context menu → click *Sign File*



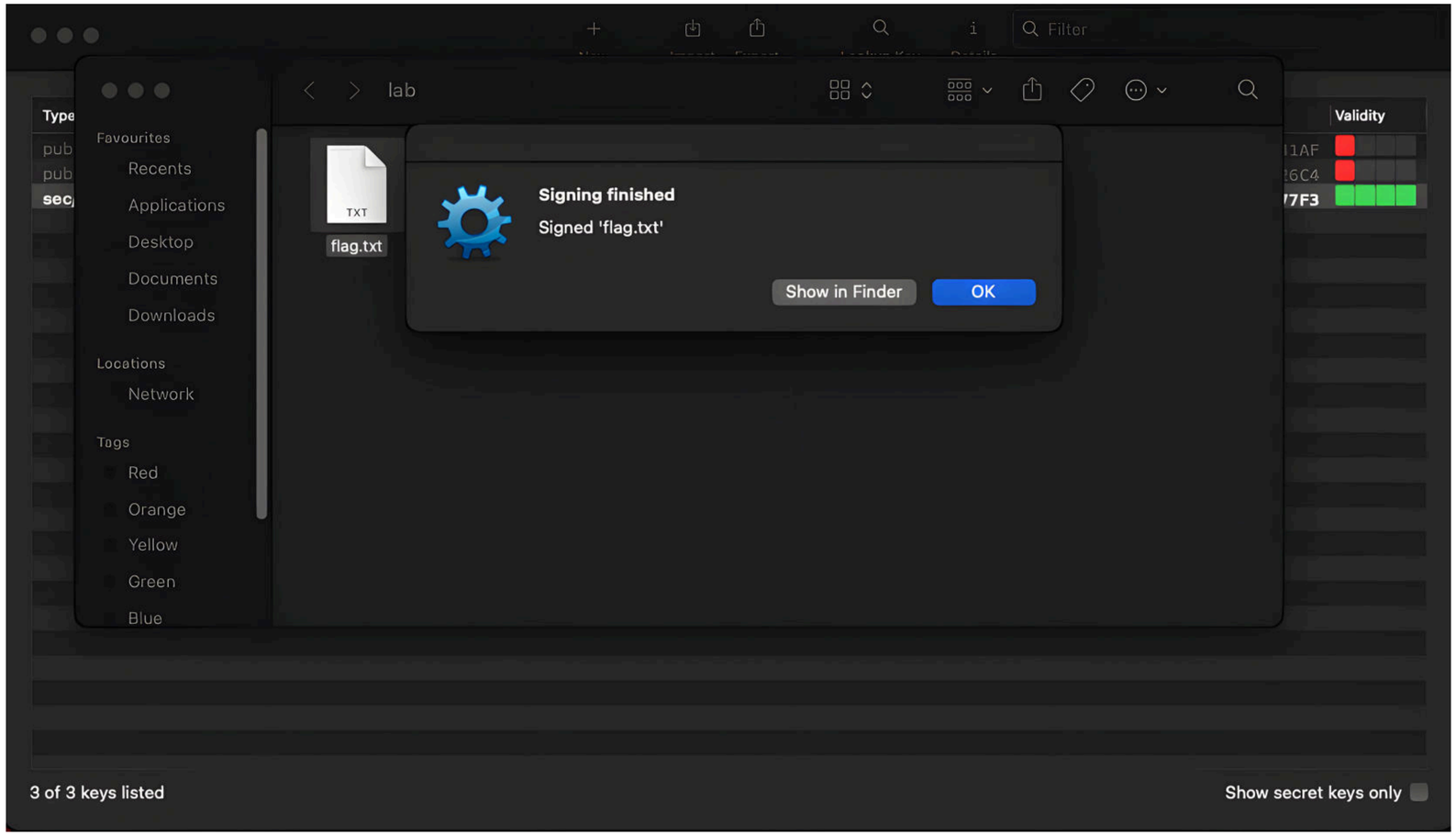
gpgtools: signing a file

Enter the passphrase chosen when creating the key pair to finalize the signing:



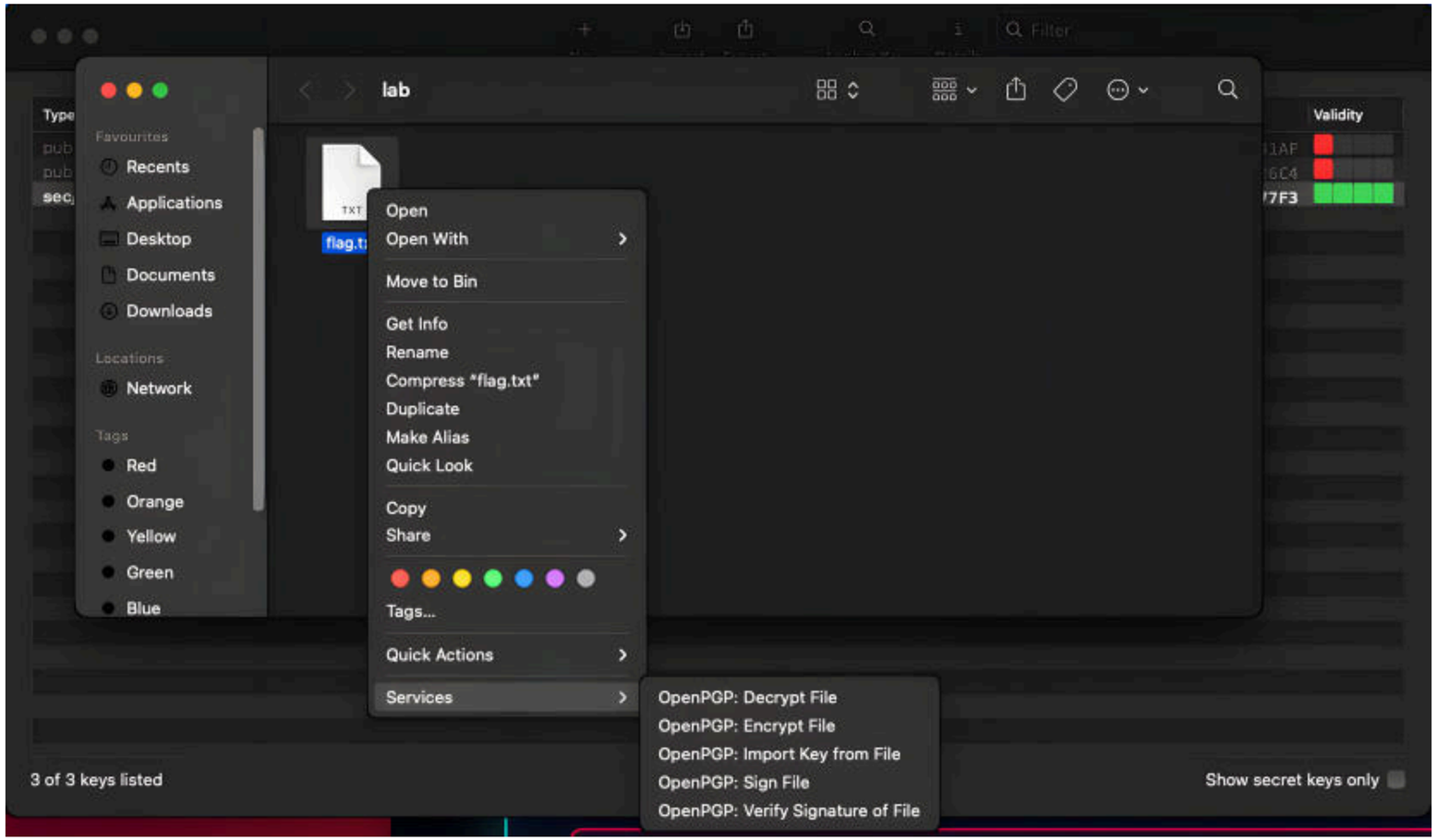
gpgtools: signing a file

Success screen:



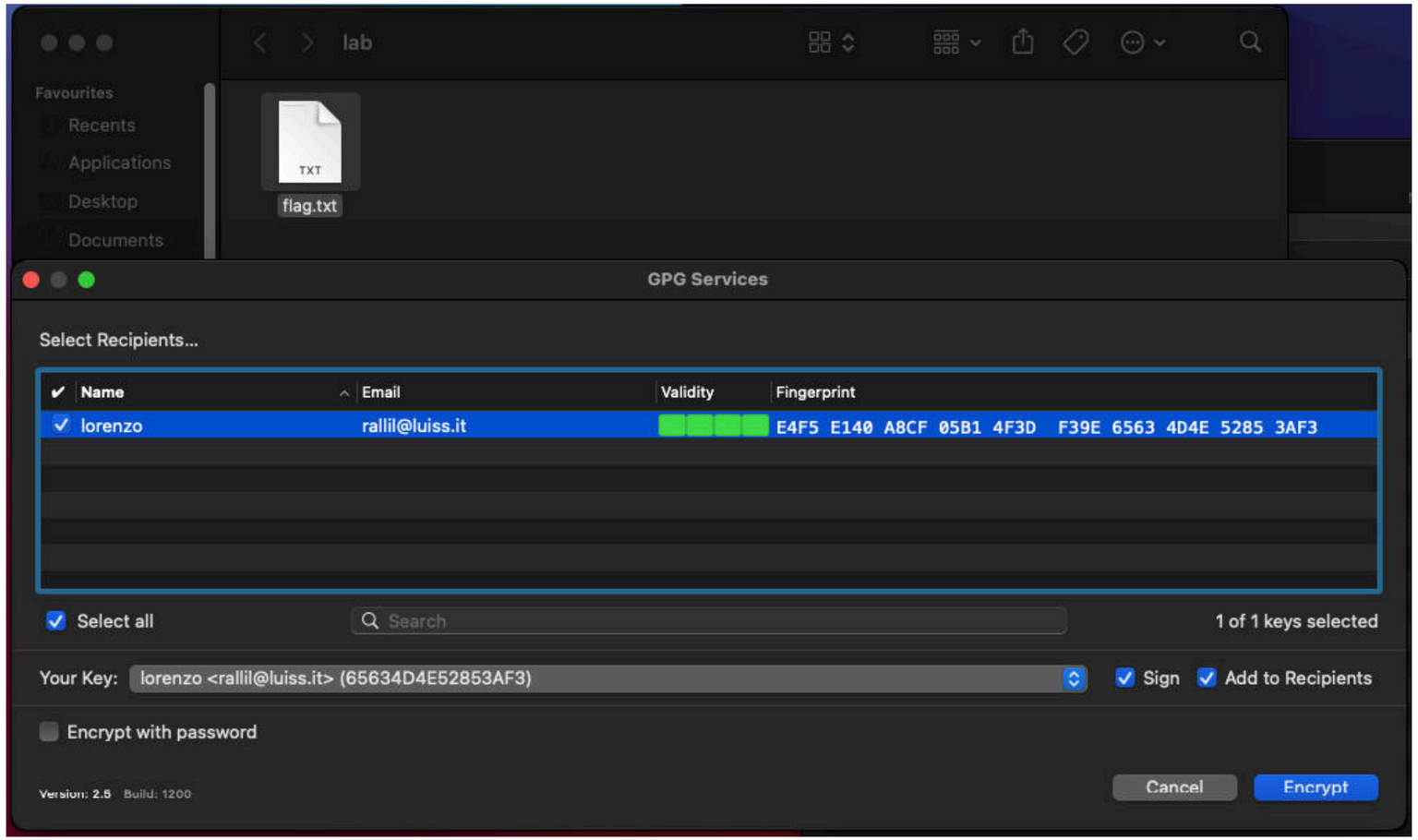
gpgtools: encrypting a file

Right click the file to sign to spawn the context menu → click *Encrypt File*:



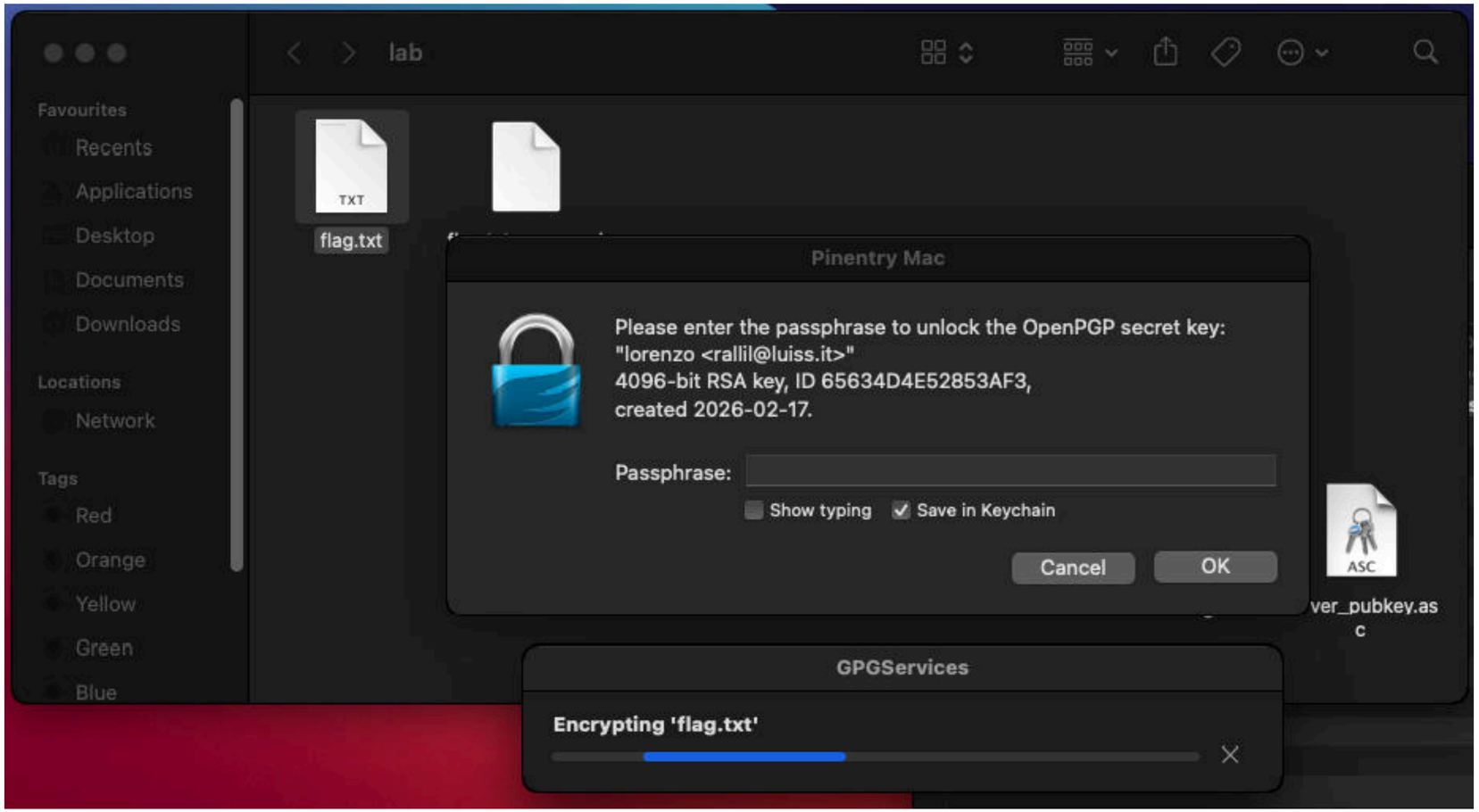
gpgtools: encrypting a file

Select the key to use to encrypt the file:



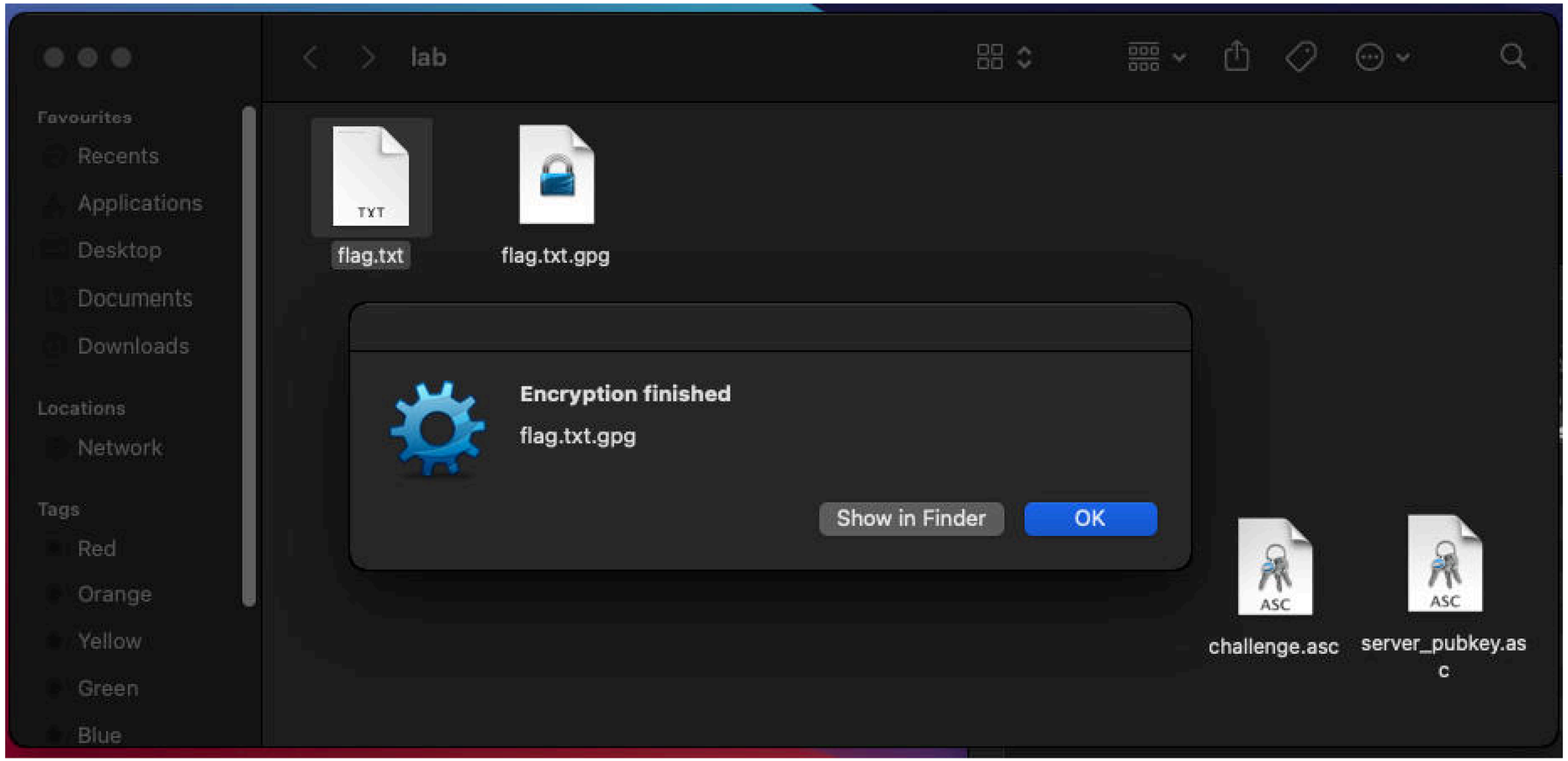
gpgtools: encrypting a file

Enter the password chosen at key pair creation:



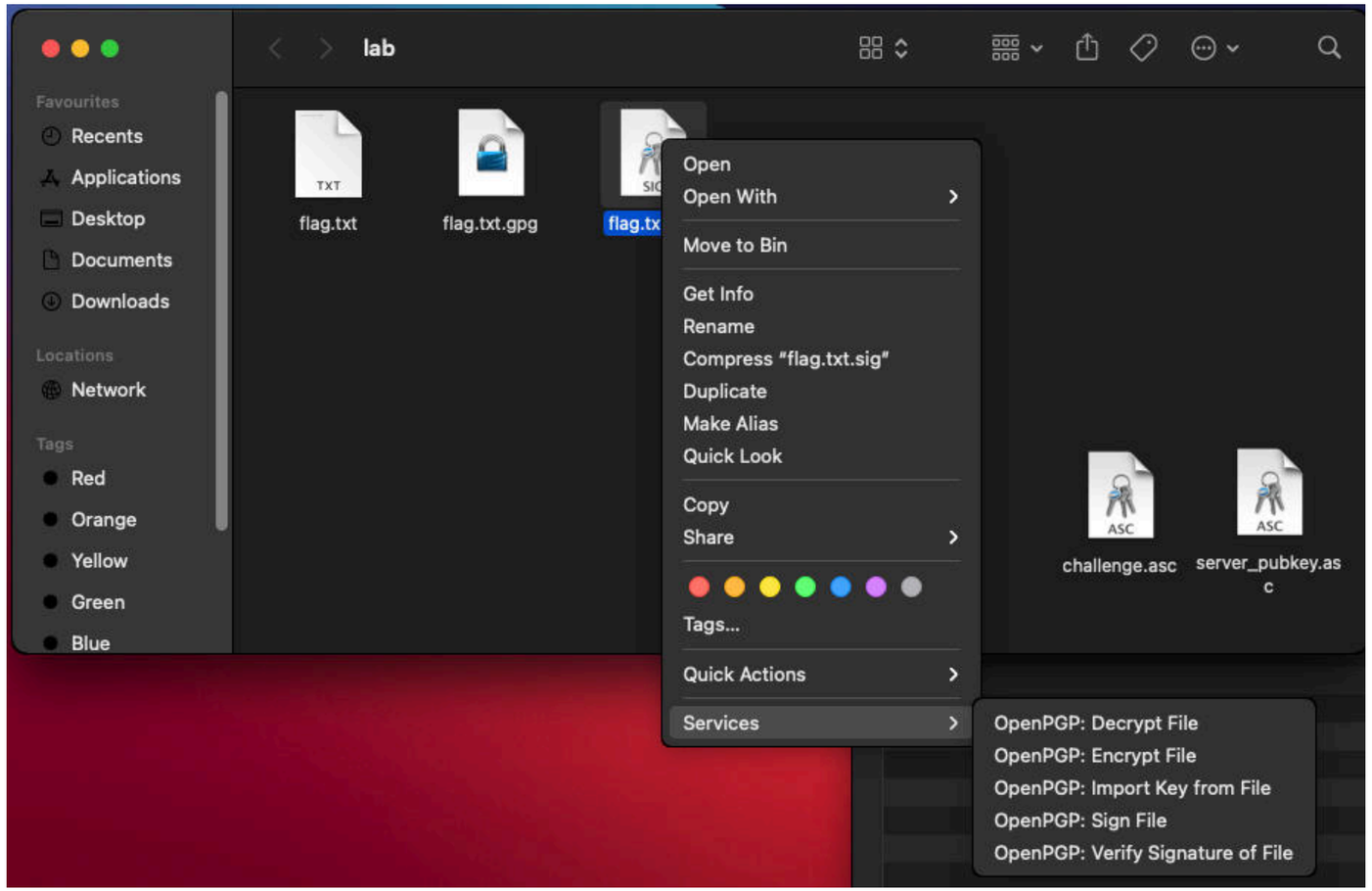
gpgtools: encrypting a file

Success screen:



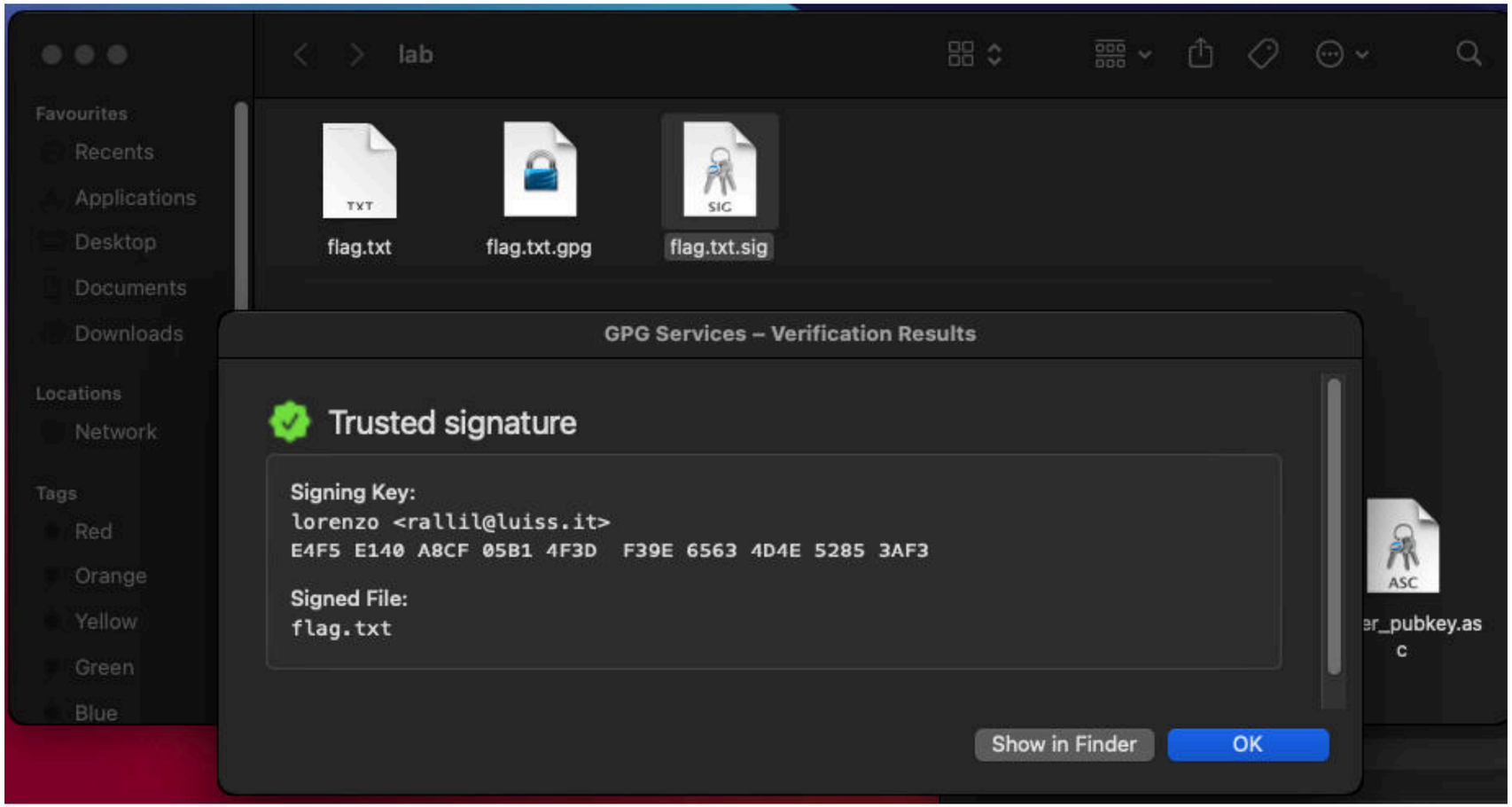
gpgtools: verifying a signature

Right click the file to verify to spawn the context menu → click *Verify Signature of file*:



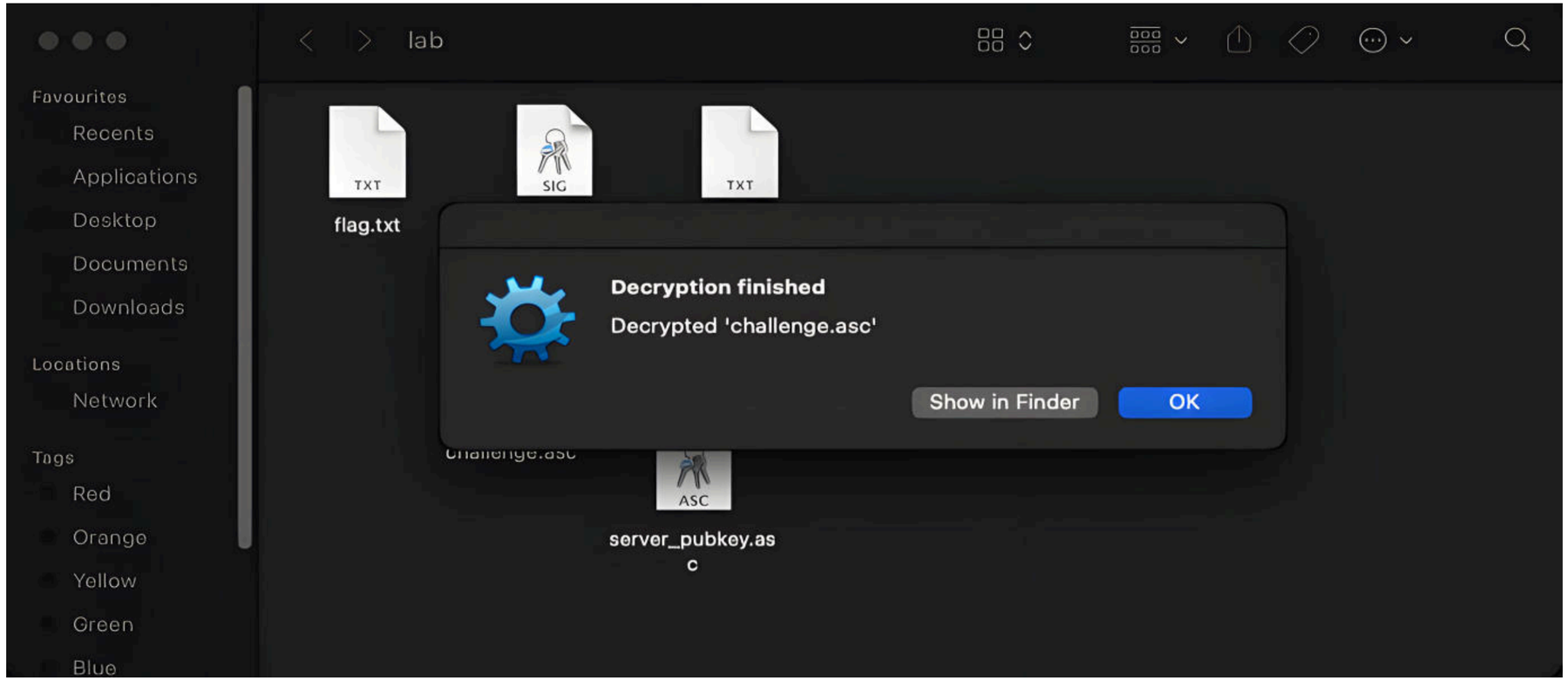
gpgtools: verifying a signature

Success screen:



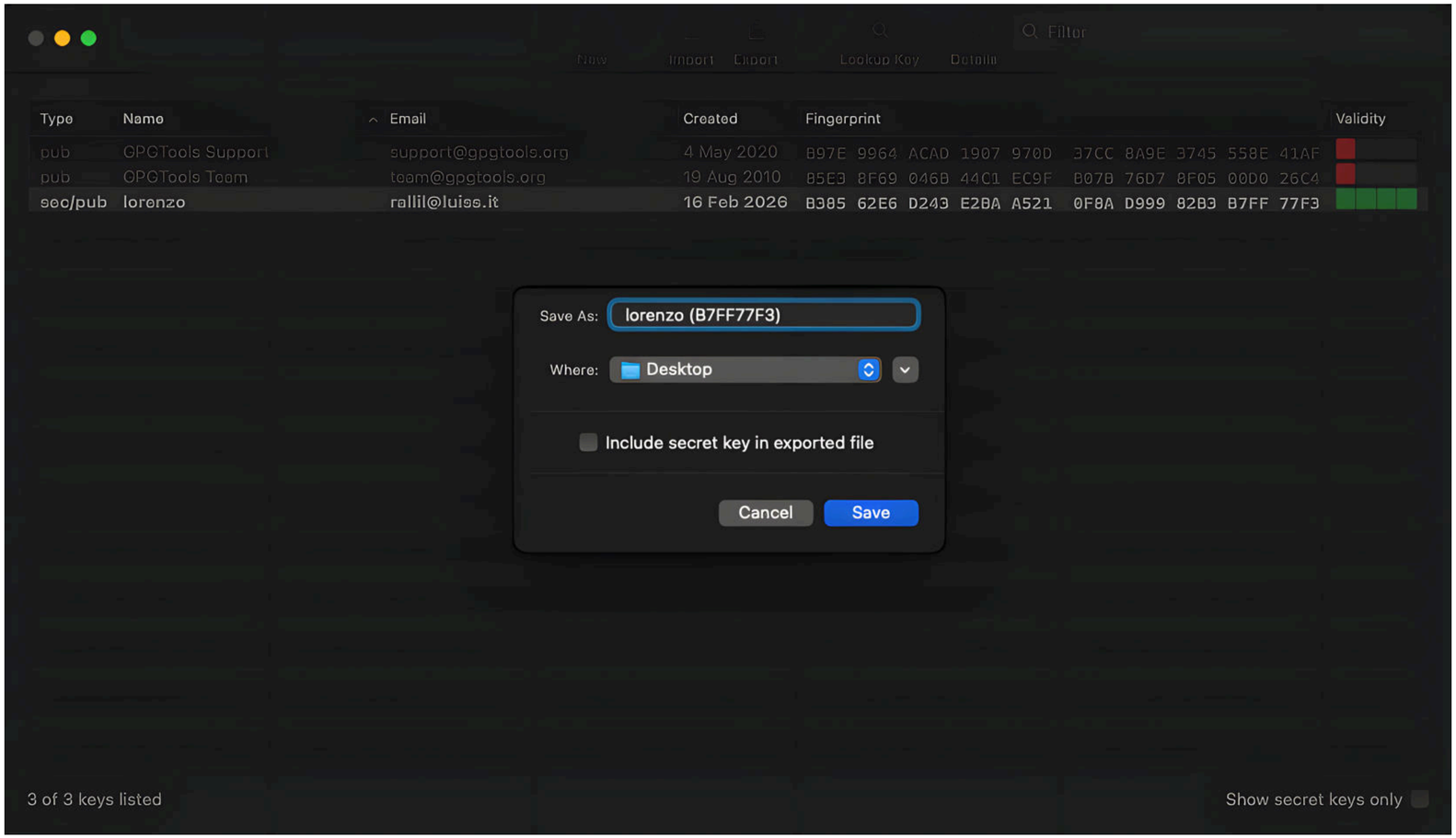
gpgtools: decrypting a file

Right click the file to decrypt to spawn the context menu → click *Decrypt File*.



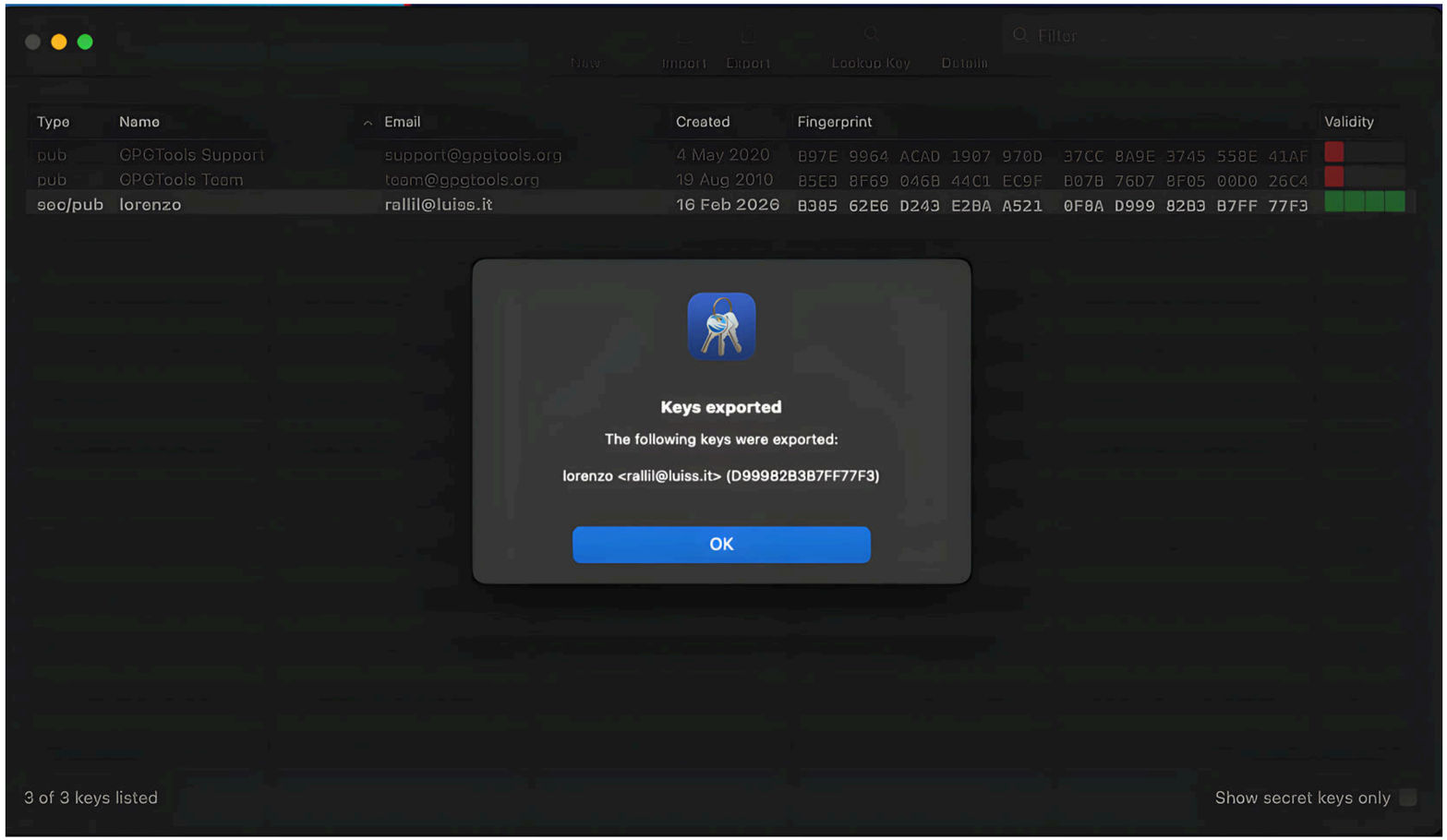
gpgtools: exporting a public key

Click *Export* in the action toolbar → choose which key to export and where to:



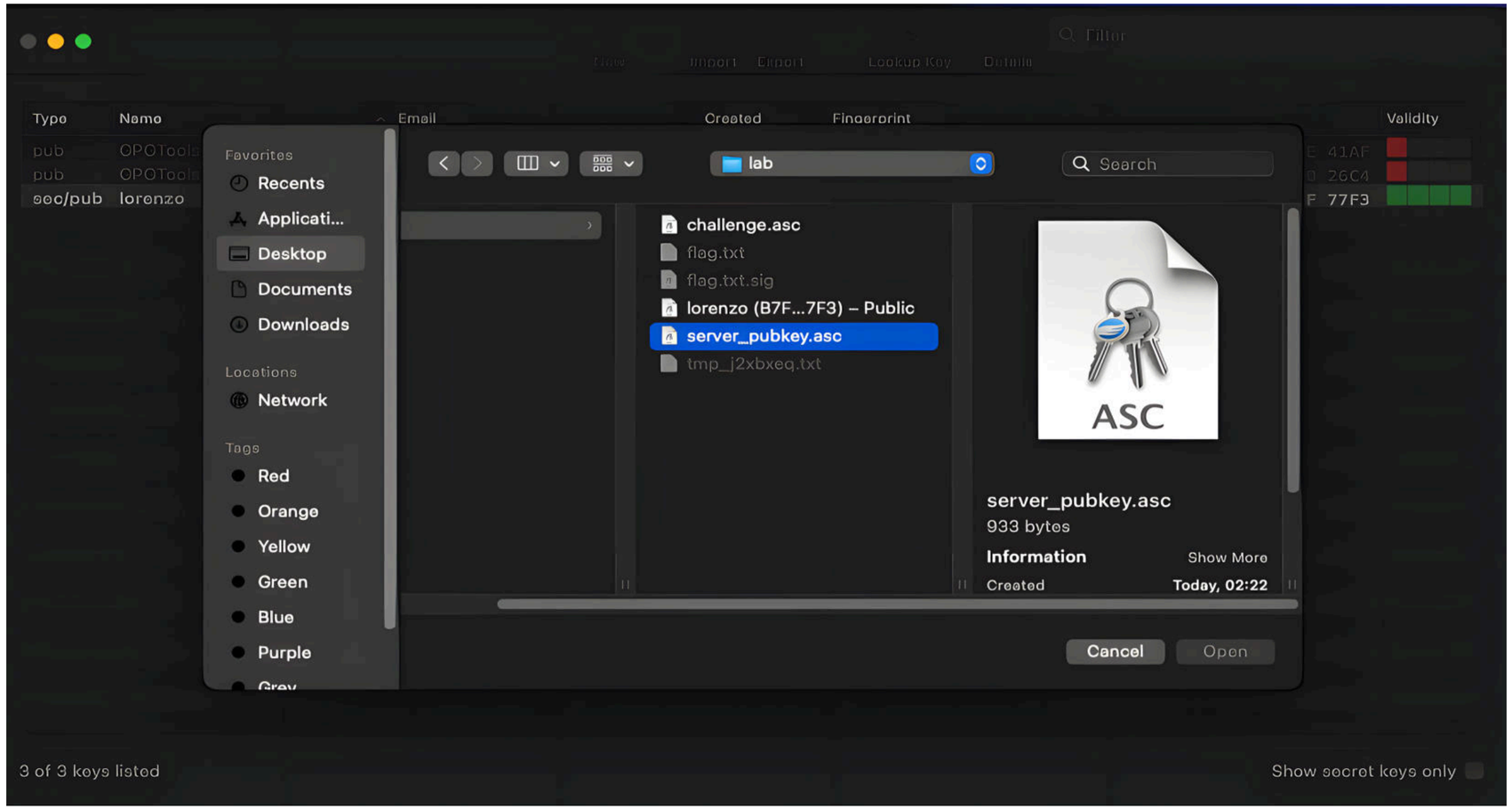
gpgtools: exporting a public key

Success screen:



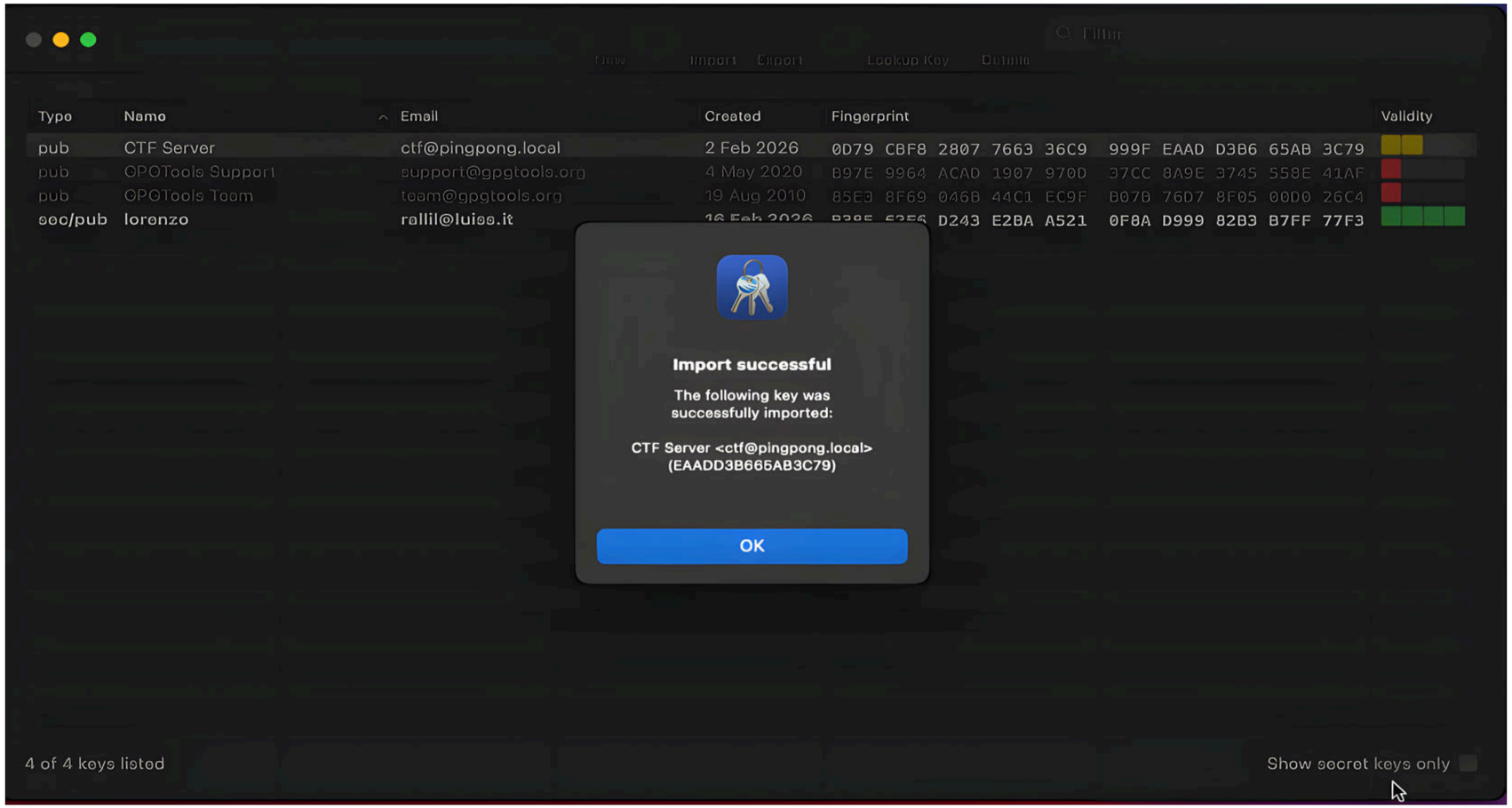
gpgtools: importing a public key

Click *Import key from file* in the action toolbar → choose which key to import from the pop up menu:



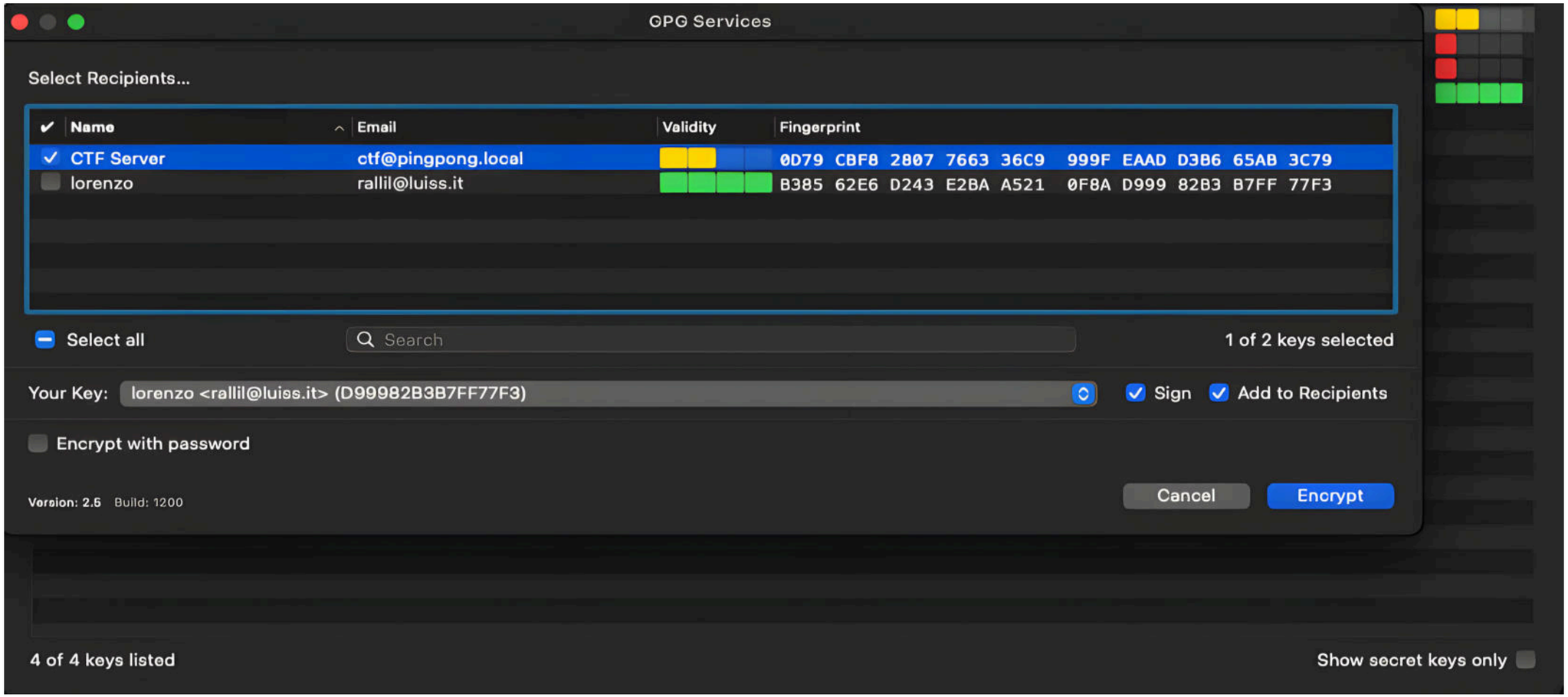
gpgtools: importing a public key

Success screen:



gpgtools: encrypting a file with the imported public key

Right click the file to sign to spawn the context menu → click *Encrypt File* → pick the imported key:



Time to Cook

SharedFortune

Crypto Challenge 10

SharedFortune - Reconnaissance

SharedFortune (500 points) crypto

Different games, different rules… but the house likes to reuse what already works.

https://crypto10.webhack.it

Flag

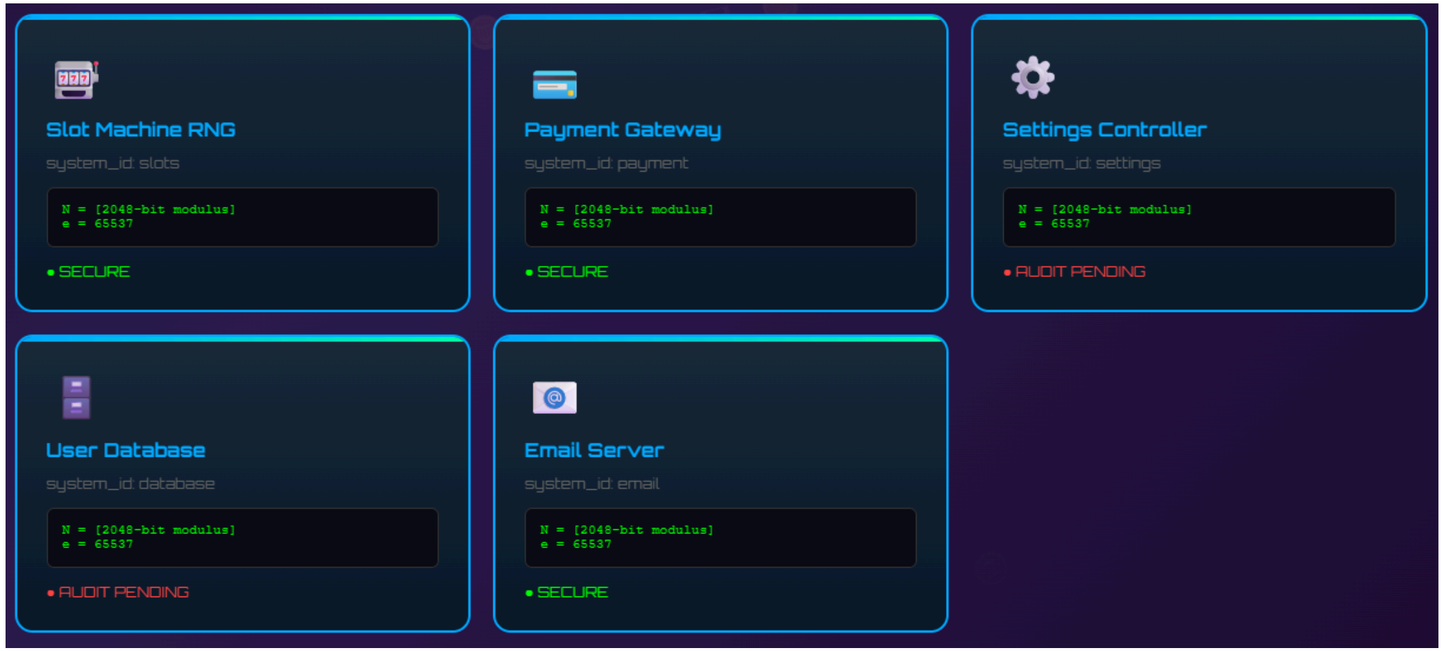
Submit

What can we observe? The challenge...

- provides a casino-themed web interface with sections for *Slots*, *Shop*, *Systems*, and *Settings*;
- includes a *Download All Public Keys* button in the *Systems* section;
- requires a secret to unlock *Settings*, and coins to buy the flag in the *Shop*.

SharedFortune - Hypothesis

The name of the challenge hints at RSA moduli **sharing** prime factors.



Two of the systems leveraged by the casino are yet to be audited...

CryptoBattle

Crypto Challenge 12

CryptoBattle - Reconnaissance

CryptoBattle (500 points) crypto

Gotta reverse 'em all!

https://crypto12.webhack.it

Flag

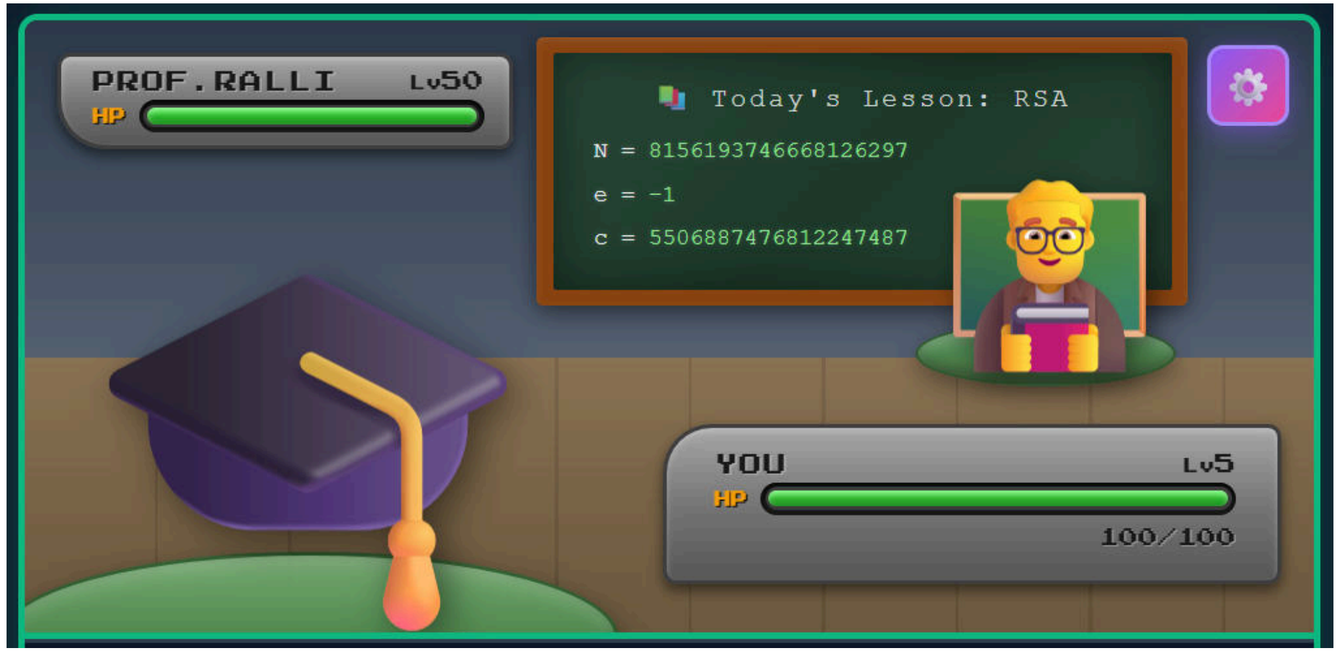
Submit

What can we observe? The challenge...

- displays RSA parameters (n, e, c) on a chalkboard;
- uses a turn-based battle system with player and professor HP;
- provides a cheat console that accepts a password.

CryptoBattle - Hypothesis

There are RSA parameters and a ciphertext displayed on the chalkboard within the battle screen.



The public exponent e is **suspiciously negative**...

SlidingIntoDHs

Crypto Challenge 13

SlidingIntoDHs - Reconnaissance

SlidingIntoDHs (500 points) crypto

End-to-end encrypted chats...except for the part in the middle.

https://crypto13.webhack.it

Flag

Submit

What can we observe? The challenge...

- implements a Diffie-Hellman powered chat between two professors;
- allows us to act as a Man-in-the-Middle during the key exchange;
- reveals all public parameters, and provides the intercepted encrypted message.

GPGPingPong

Crypto Challenge 14

GPGPingPong - Reconnaissance

GpgPingPong (500 points) crypto

Sometimes to get something, you just need to ask poletely!

https://crypto14.webhack.it

Flag

Submit

What can we observe? The challenge...

- displays a server-generated message that we must sign;
- requires uploading both a signature file and a public key file;